



University of St.Gallen

Institute of Computer Science

Performance Mining

Process Mining

Session 5



From insight to impact.

Outline

Session 1: Introduction

- *Event Logs, Event Log Inspection, First Exploration*

Session 2: Initial Event Log Analysis

- *Initial Analysis, Visualizations, Log Enrichments*

Session 3: Process KPIs

- *Process Outcomes, Dashboards*

Session 4: Behavioral Analysis and –Modeling

- *Variants, Process Maps, BPMN, Petri Nets*

Session 5: Performance Mining

- *Rework, Load and Demand, Bottlenecks*

Session 6: Conformance Checking

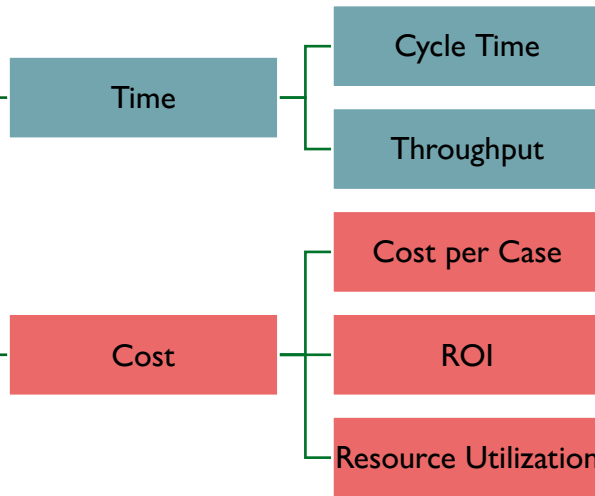
- *Business Rules and Normative Process Models*

Session 7: Deviance Diagnostics

- *Explanations and Rule Mining*

Assignment 1 Part 2 due

Process Performance



Total time taken to complete a process from start to finish.

The number of cases processed within a given time period.

Return on Investment

Percentage of work that needs to be redone due to errors.

Net Promoter Score (Willingness of customer to recommend service.)

Outline

1. Rework
2. Load and Demand
3. Cycle Time Analysis
4. Shift Left Analysis

Rework Analysis

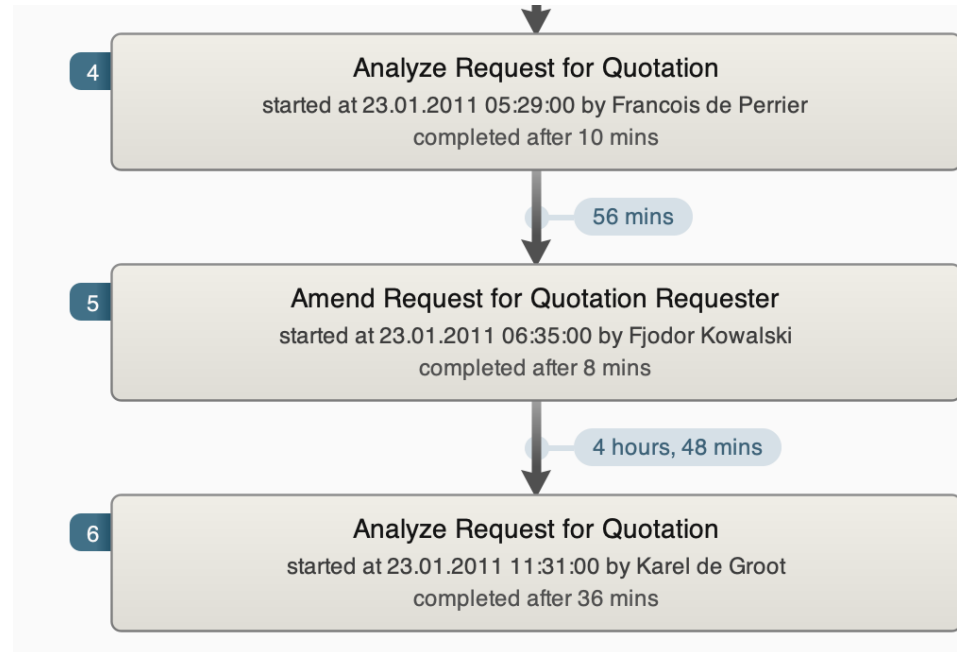
Rework (= multiple occurrences of the same activity *in a case*)

- can be self loop: A, A, ... or
- short loop A, B, A, B ... or
- longer loop A, B, C, D, A, ...

Repetition within a case may indicate

- first instance of activity was not successful and needed to be revisited
 - e.g. because instructions were not clear to the user (e.g. usability issue)

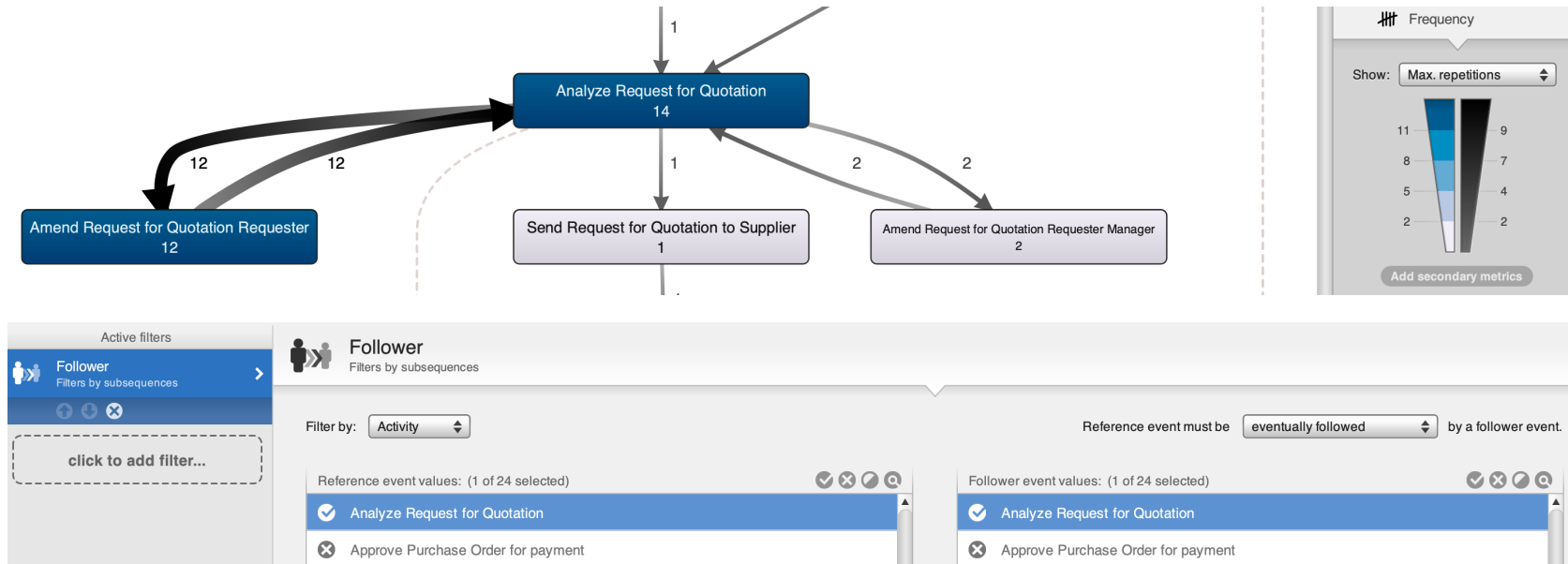
A loop in the (full) DFG is necessary but not sufficient for rework being present in the log



Rework Analysis (e.g. in Disco)

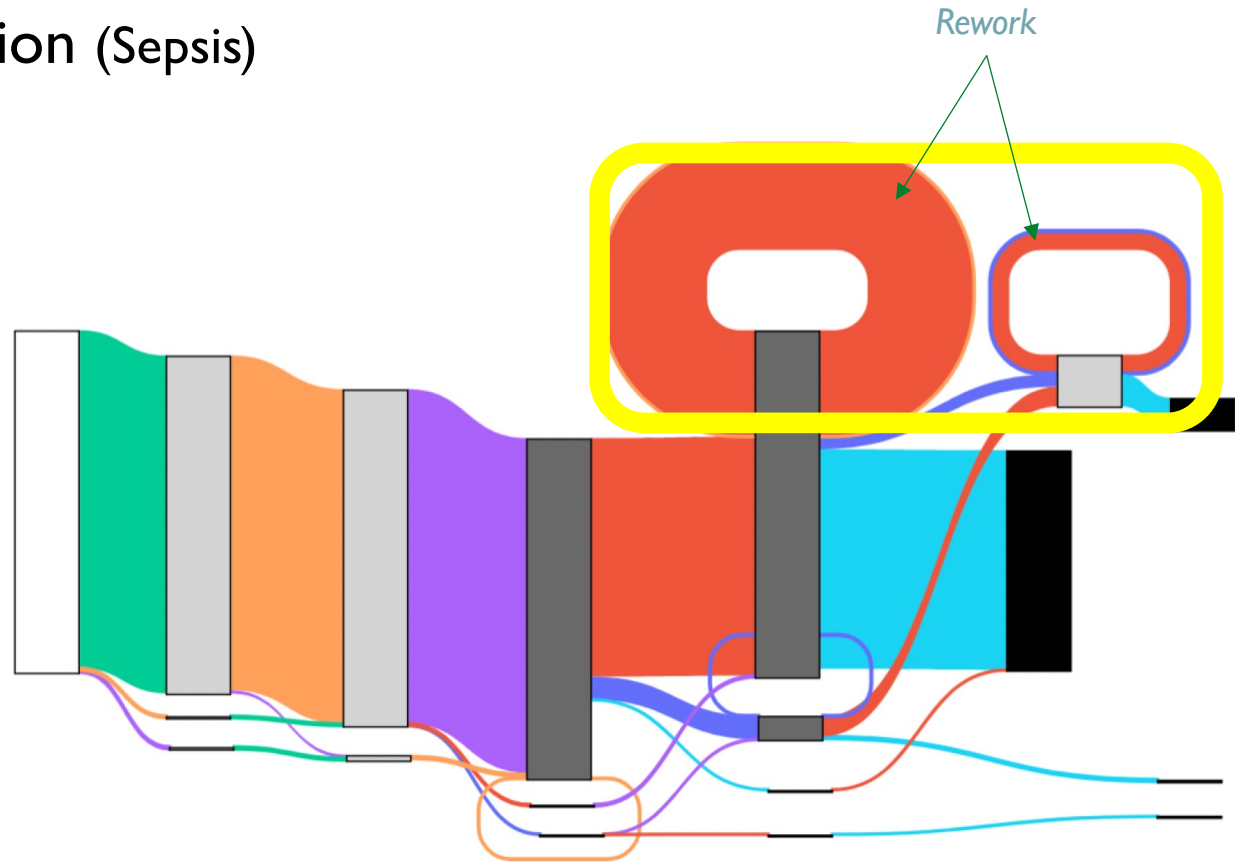
Recall the filter to reduce graph complexity

- Use *max repetition* annotation of DFG to find activities that are repeated
- Use follower filter to filter variants that contain the repeated activity (using *eventually follows*)
- Inspect variants / cases for details

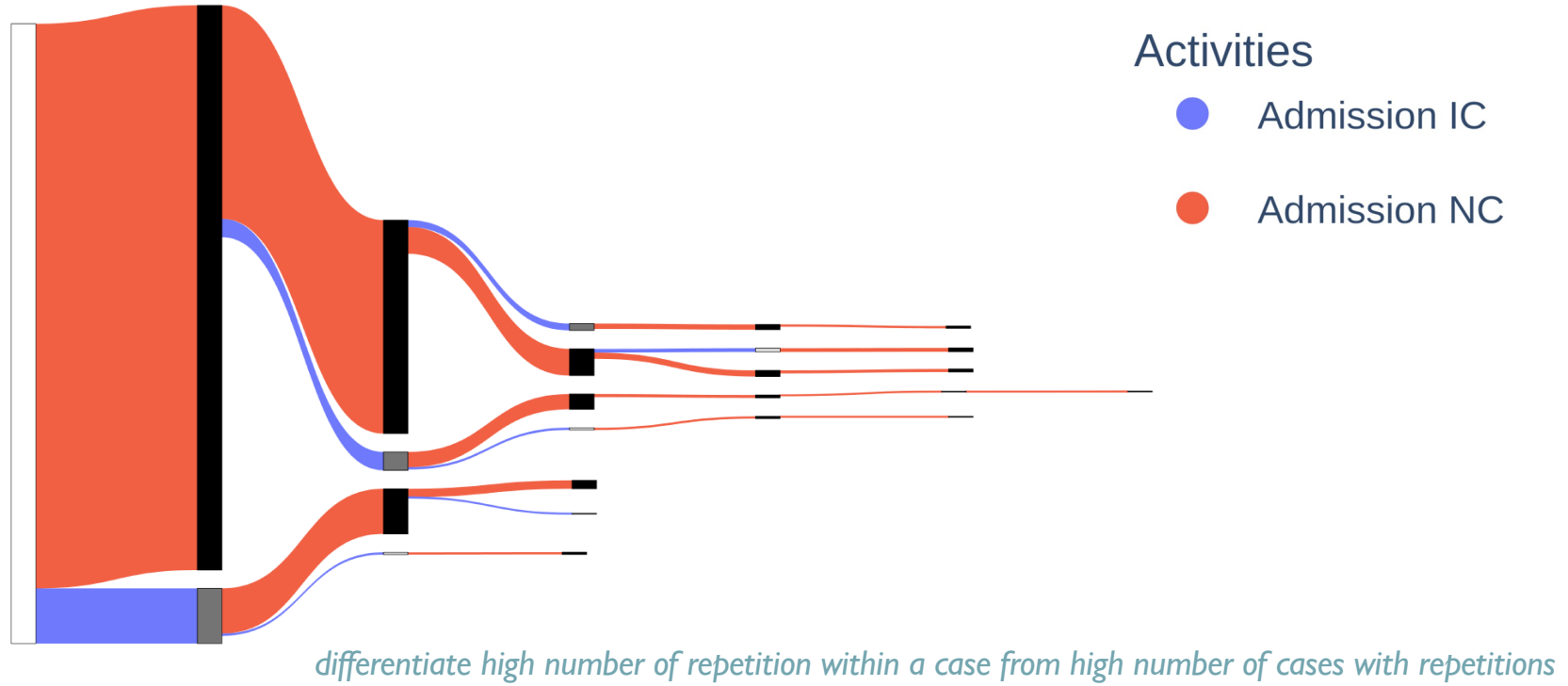


LCM Set Abstraction (Sepsis)

- Admission IC
- Admission NC
- ER Registration
- ER Sepsis Triage
- ER Triage
- Release



LCM Tree (Sepsis) – Unfolds Loops



Rework Analysis Playbook

Use *dedicated queries* and/or visualizations to:

- identify activities that repeat within cases
- determine the number of cases affected
- *understand the distribution* of number of repetitions per case
- do *case inspection* for typical rework cases as well for outlier rework cases
 - try to understand the reason for rework

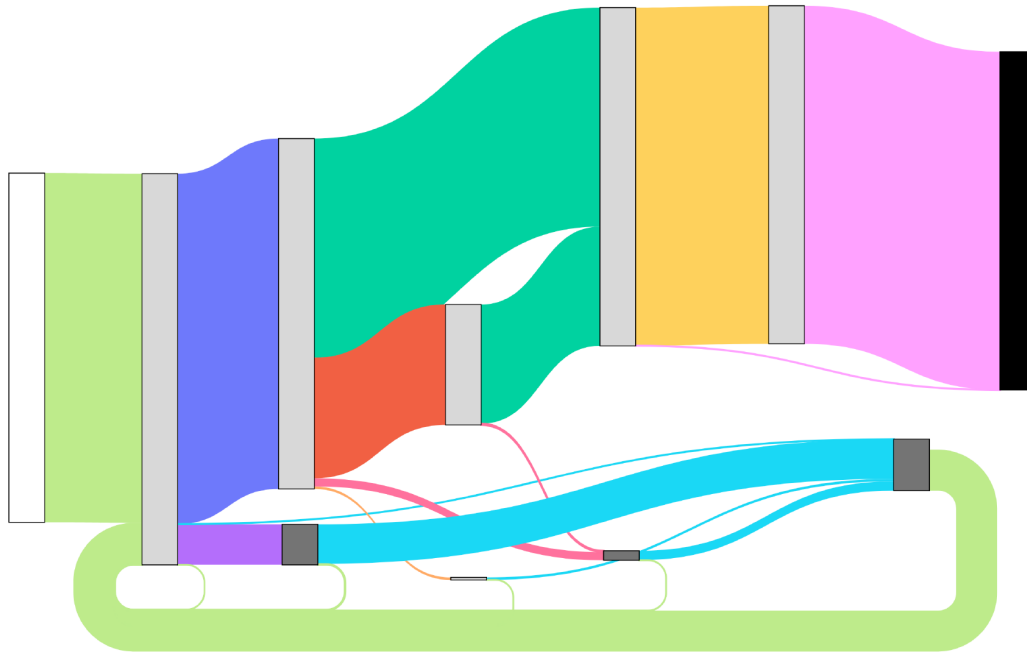
Load and Demand

Event Log: Domestic Declarations

Download the event log [here](#).

The dataset contains events pertaining to two years of travel expense claims. The various permits and declaration documents (domestic and international declarations, pre-paid travel costs and requests for payment) all follow a similar process flow.

Log Overview (Domestic Declarations)

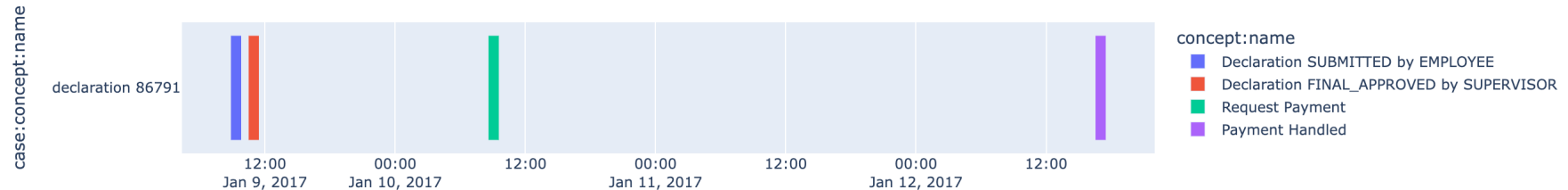


Activities

- Declaration APPROVED by ADMINISTRATION
- Declaration APPROVED by BUDGET OWNER
- Declaration FINAL_APPROVED by SUPERVISOR
- Declaration REJECTED by ADMINISTRATION
- Declaration REJECTED by BUDGET OWNER
- Declaration REJECTED by EMPLOYEE
- Declaration REJECTED by SUPERVISOR
- Declaration SUBMITTED by EMPLOYEE
- Payment Handled
- Request Payment

Single Case Timeline (*time-extended case inspection*)

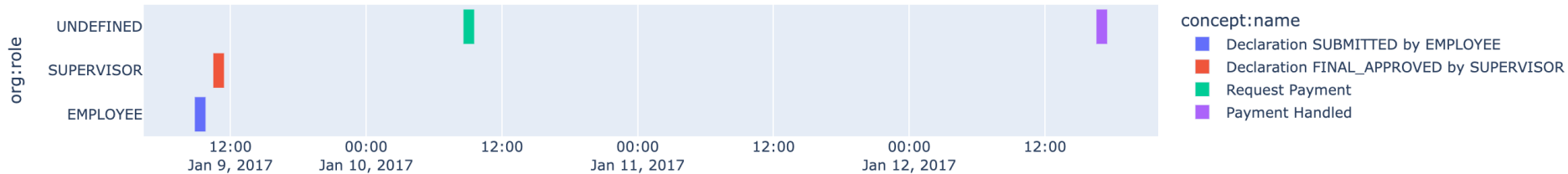
```
filtered_event_log = event_log[event_log['case:concept:name'] == 'declaration 86791']
bar_size = pd.Timedelta(hours=1)
fig = px.timeline(filtered_event_log,
                  x_start=(filtered_event_log['time:timestamp']-bar_size),
                  x_end=filtered_event_log['time:timestamp'],
                  y='case:concept:name',
                  color='concept:name', height=250)
fig.show()
```



x-axis data must be of type datetime in px.timeline

Gantt Chart (adding role or resource as swimlane)

```
filtered_event_log = event_log[event_log['case:concept:name'] == 'declaration 86791']
bar_size = pd.Timedelta(hours=1)
fig = px.timeline(filtered_event_log,
                 x_start=(filtered_event_log['time:timestamp']-bar_size),
                 x_end=filtered_event_log['time:timestamp'],
                 y='org:role',
                 color='concept:name', height=250)
fig.show()
```



Exercise (*sepsis_v3*)

Draw case *H* of the *Sepsis* log as a Gantt chart using *org:group* as swim lanes.

Compare with the tabular data for case *H*.

Write down some interesting observations.

Dotted Chart (absolute time)



What can we see?

```
from event_enricher import create_case_log, event_add_ordered_case_id
```

```
case_log = create_case_log(event_log)
```

```
event_log = event_add_ordered_case_id(event_log, case_log)
```

```
filtered_case_log = case_log.sort_values(by='start_time') ← sort by arrival time
```

```
filtered_case_log = filtered_case_log[:22] ← select first 22 cases
```

```
filtered_event_log = event_log[event_log['case:concept:name'].isin(filtered_case_log.index)]
```

```
bar_size = pd.Timedelta(hours=8) ← adjust bar width
```

```
fig = px.timeline(filtered_event_log,
```

```
    x_start=(filtered_event_log['time:timestamp']+bar_size),
```

```
    x_end=filtered_event_log['time:timestamp'],
```

```
    y='ordered_case_id',
```

```
    color='concept:name',
```

```
    height=500) ← adjust figure height such that you can see enough
```

```
fig.update_yaxes(categoryorder='category descending')
```

```
fig.update_layout(xaxis={'side': 'top'}, yaxis={'side': 'left'})
```

```
fig.show()
```

Dotted Chart (absolute time)



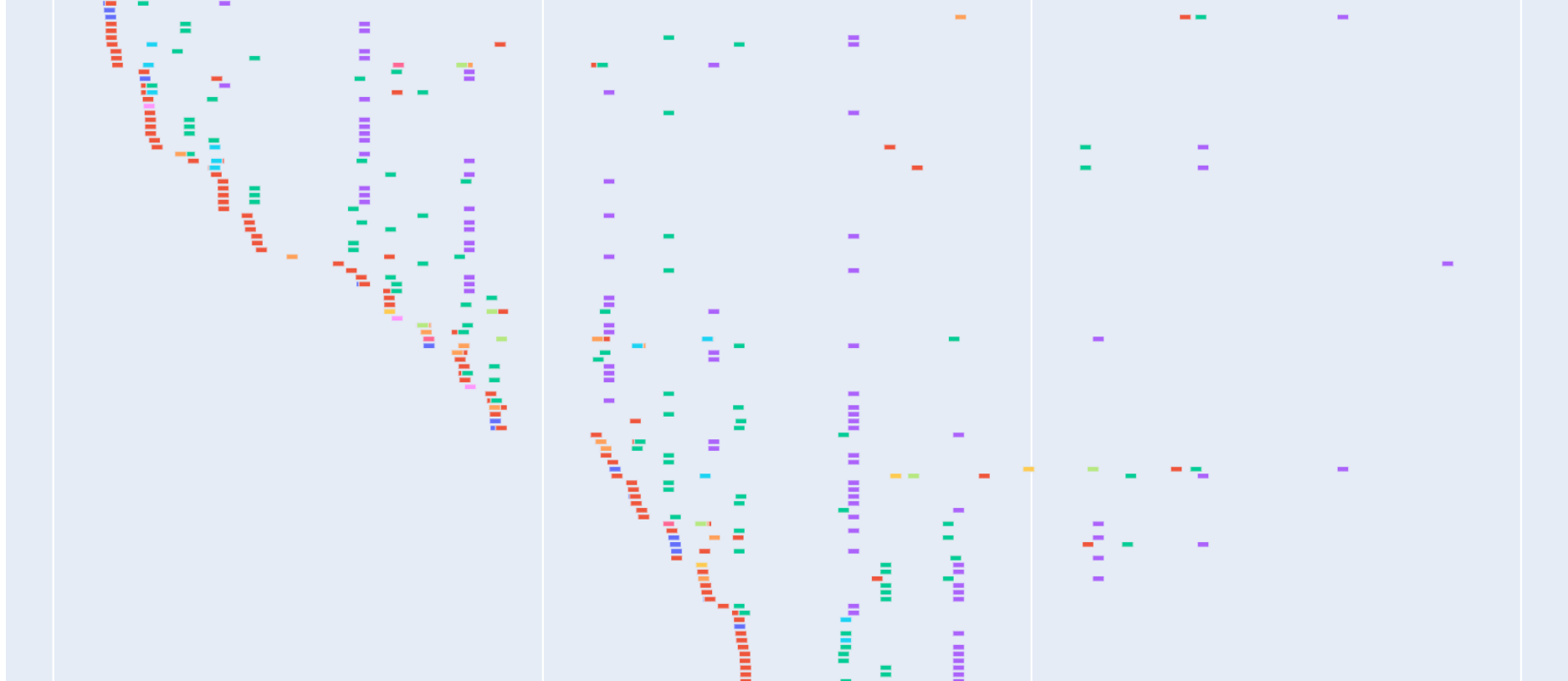
100 cases

Jan 8
2017

Jan 22

Feb 5

Feb 19



Exercise (*domestic_declarations*)

Visualize the occurrence of *Payment Handled* in the year 2017 over absolute time. Does it strengthen or weaken the hypothesis that this activity is batched?

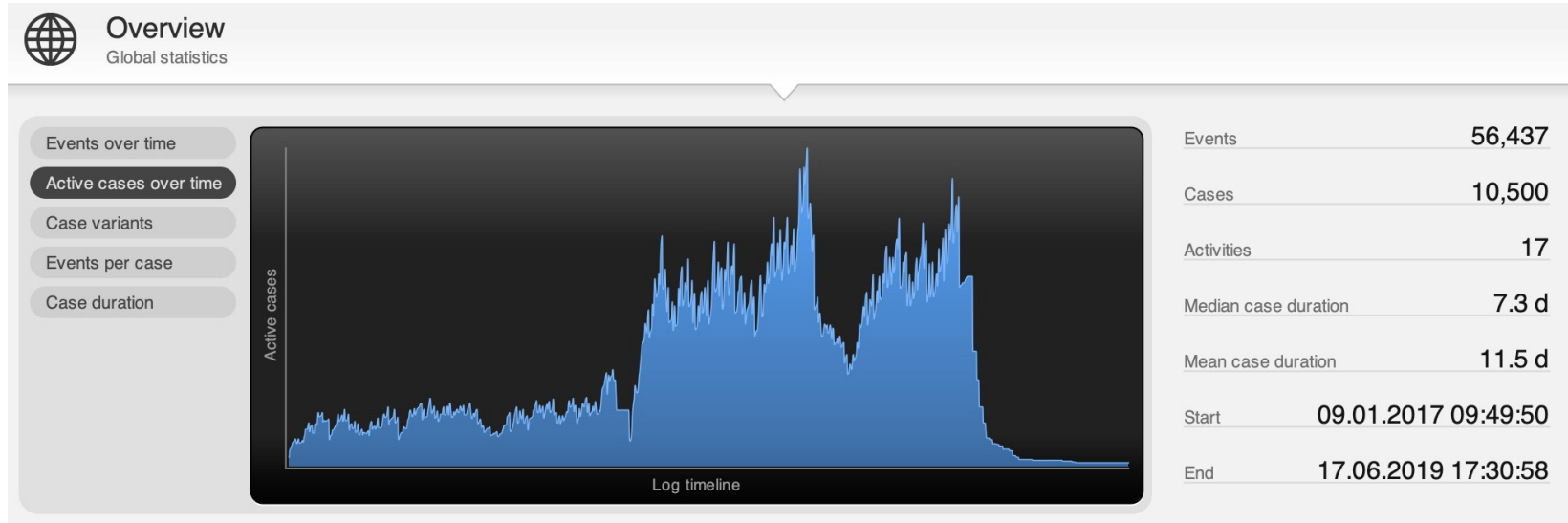
TL;DR

The absolute time dotted chart allows us to discover within a chosen time frame *indications* of the following:

- case variability (sequential variability and timing)
- load density, also specific to selected events
- patterns of batching (across cases)
- outliers (to some extent, this will improve in the relative time dotted chart)

modulo the number of cases we are able to fit into the chart.

Absolute Time Histograms (*Disco, Domestic Declarations*)

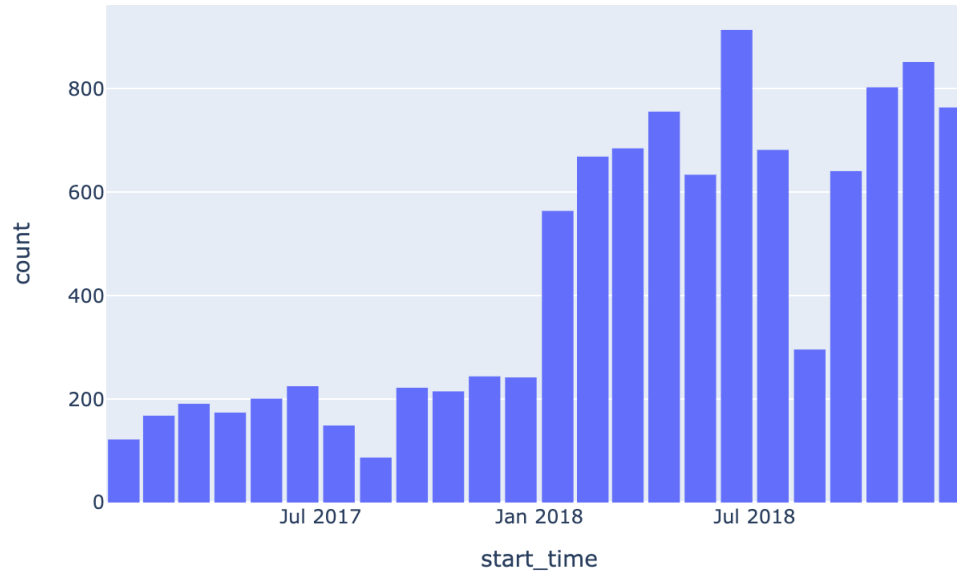


Interesting absolute time metrics

- event load (*Events over time*)
- case load (*Active cases over time*)
- Missing in Disco: *Case arrival*

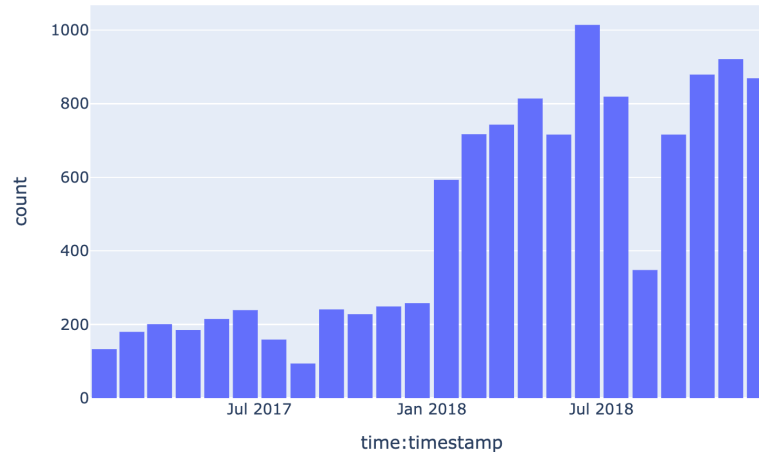
Case Arrival

```
fig = px.histogram(case_log, x='start_time', nbins=26, width=700)
fig.update_layout(bargap=0.1)
fig.show()
```



Case Arrival (*alternative version*)

```
filtered_event_log = event_log[event_log['concept:name'] == 'Declaration SUBMITTED by EMPLOYEE']  
  
fig = px.histogram(filtered_event_log, x='time:timestamp', nbins=26, width=700)  
fig.update_layout(bargap=0.1)  
fig.show()
```

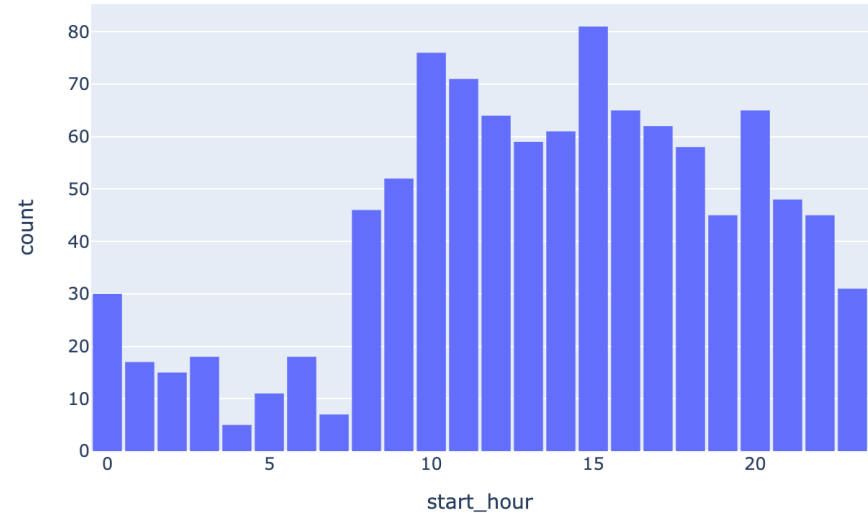


Seasonality

Understand load distribution, including

Seasonality = patterns of periodic repetition:

- specific hours of the day,
- daily,
- monthly,
- yearly,
- etc.

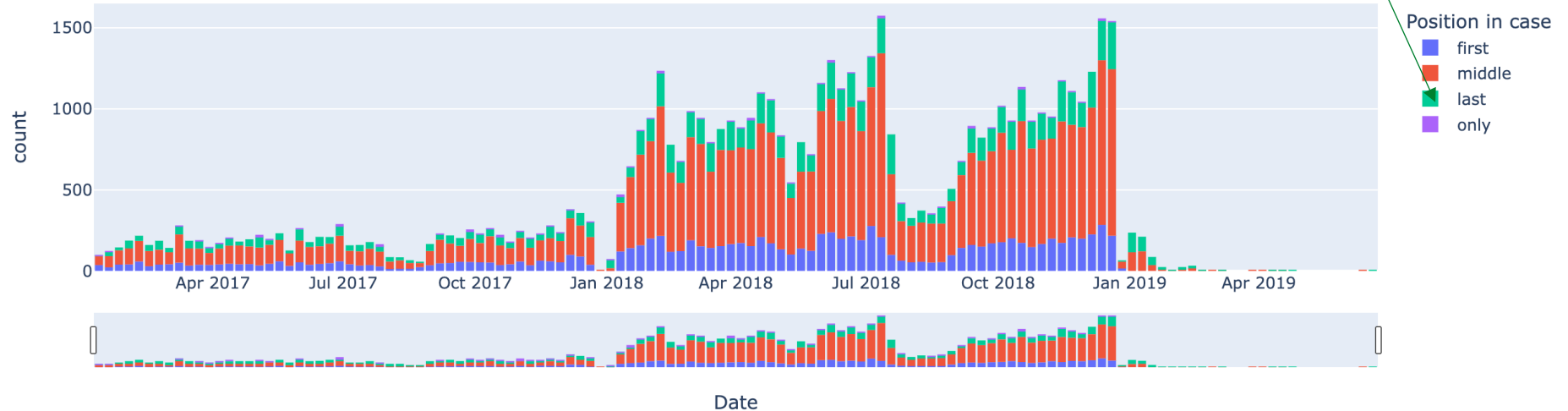


Arrival of Sepsis Triage cases over the hours of the day

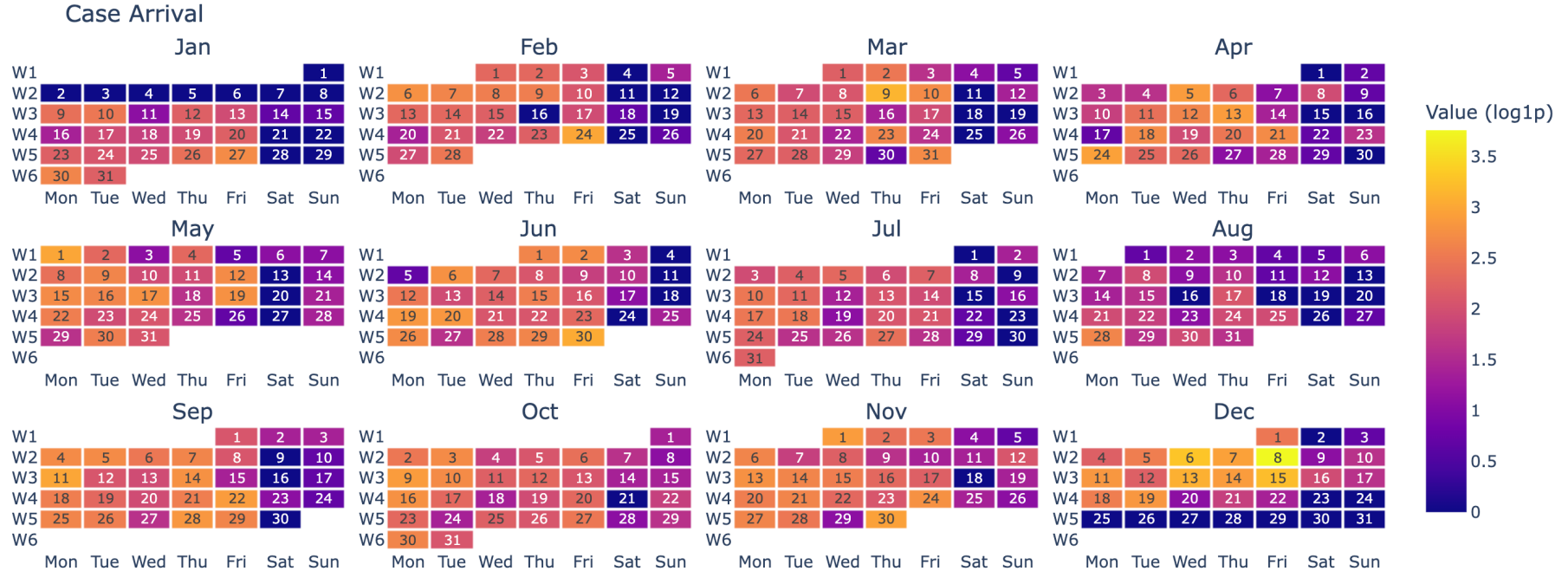
Event Load, Case Arrival and -Closure

Click to show

- arrivals only
- closures only

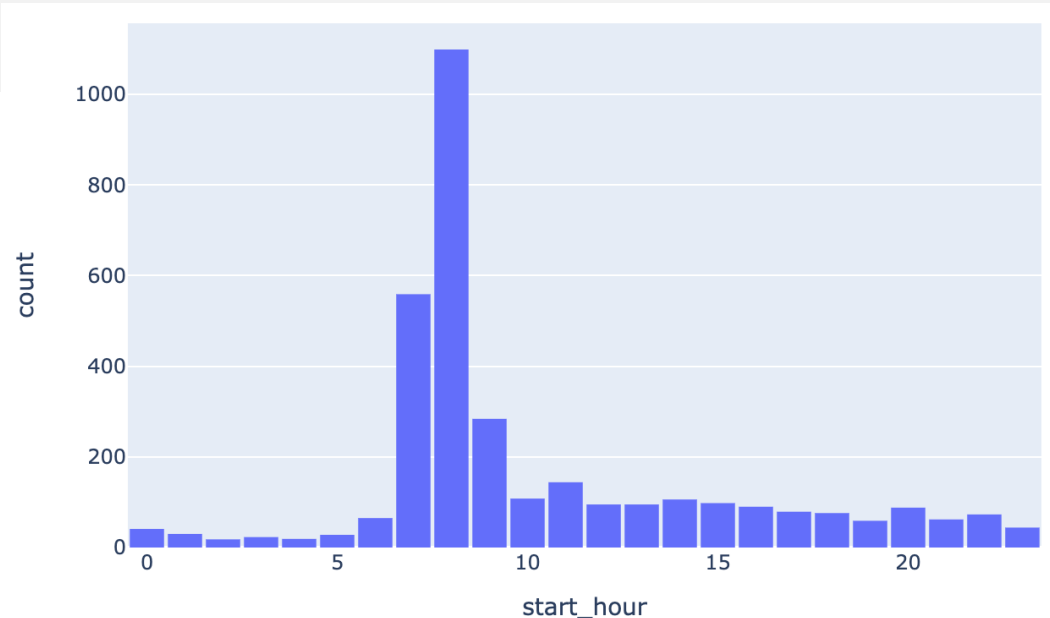


Calendar Heatmap (case arrival, domestic declarations)



Load for a Specific Activity (specialized seasonality, Sepsis log)

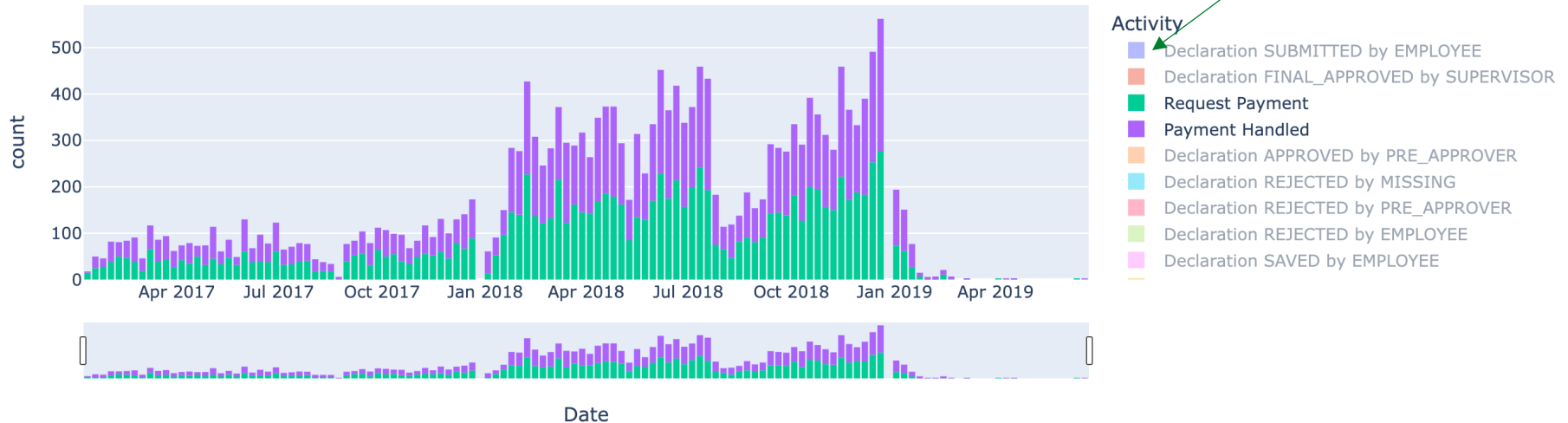
```
event_log['start_hour'] = event_log['time:timestamp'].apply(lambda x: x.hour)
filtered_event_log = event_log[event_log['concept:name'] == 'Leucocytes']
fig = px.histogram(filtered_event_log, x='start_hour', nbins=24, width=700)
fig.update_layout(bargap=0.1)
fig.show()
```



Event Load by Activity

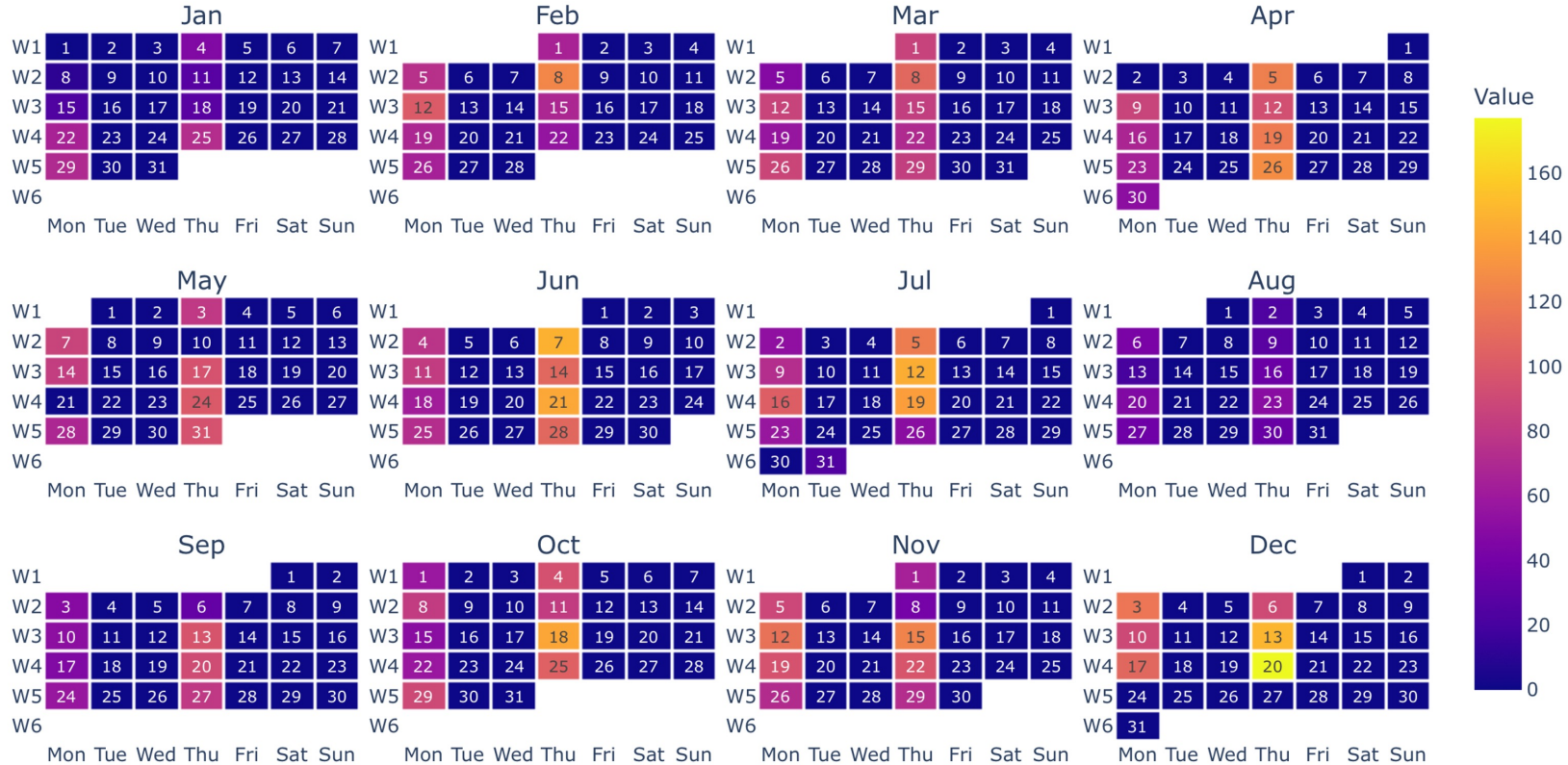
```
fig = px.histogram(event_log, x=COMPLETION_TIME, color=ACTIVITY)
fig.update_layout(bargap=0.2)
fig.update_xaxes(rangeslider_visible=True)
fig.show()
```

*Click to see load
for individual
activities*



Activity Load

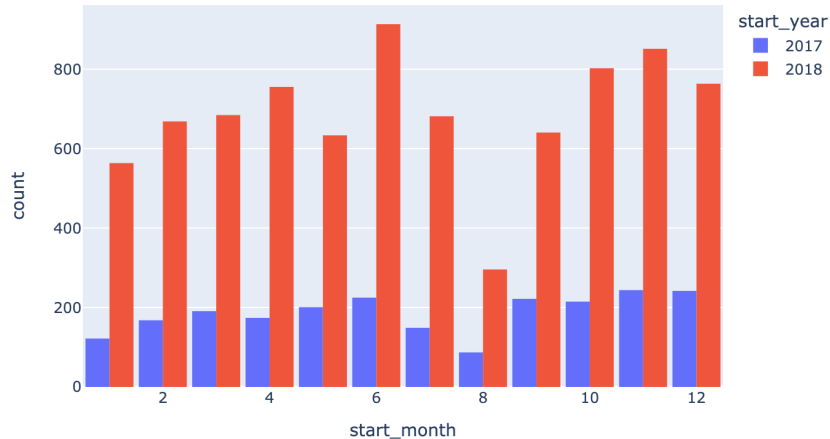
Occurrences of Payment Handled



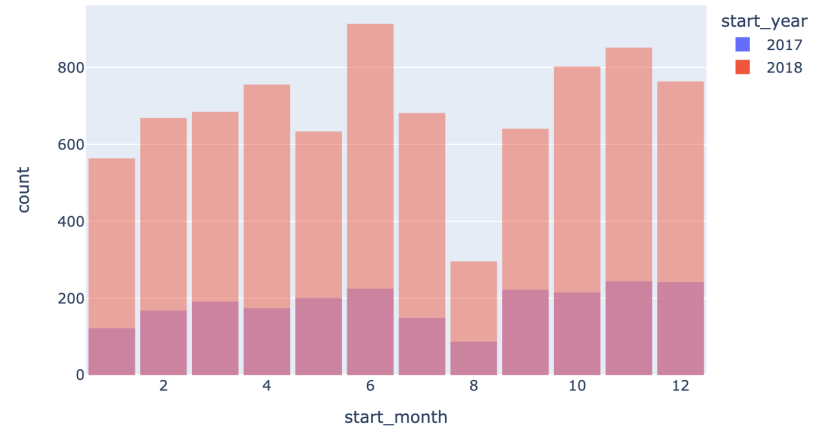
Case Arrival Drift (grouped)

```
fig = px.histogram(case_log, x='start_month', color='start_year', barmode='group', width=700)
fig.update_layout(bargap=0.1)
fig.show()
```

barmode='group'

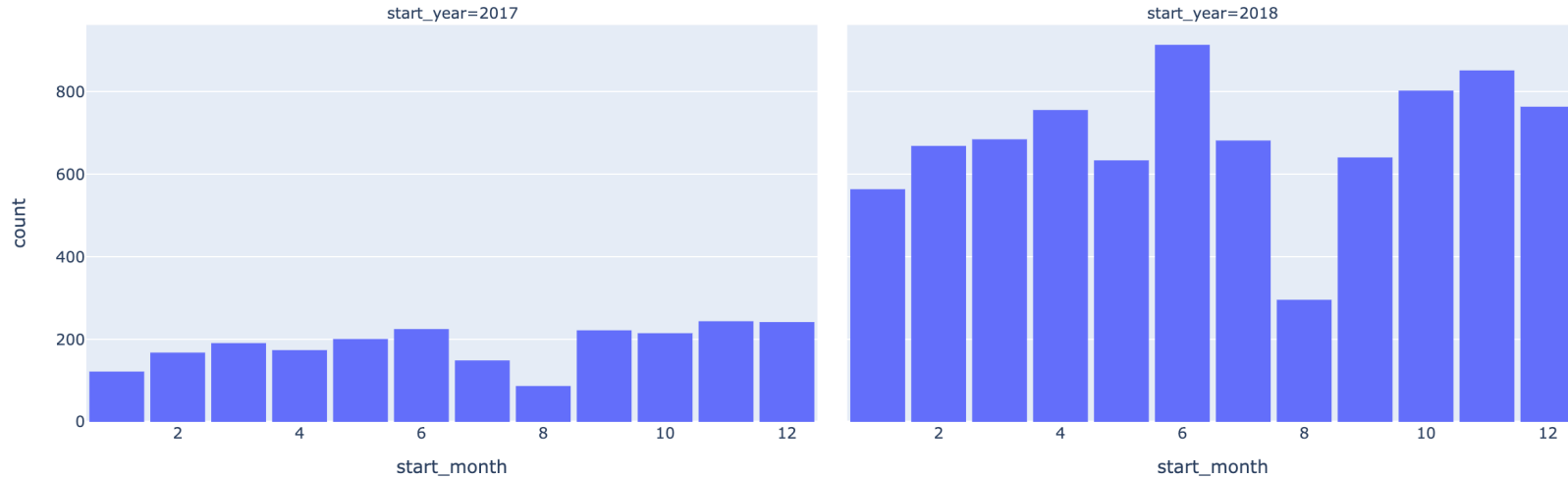


barmode='overlay'



Case Arrival Drift (alternate view, *facets*)

```
fig = px.histogram(case_log, x='start_month', facet_col='start_year')  
fig.update_layout(bargap=0.1)  
fig.show()
```



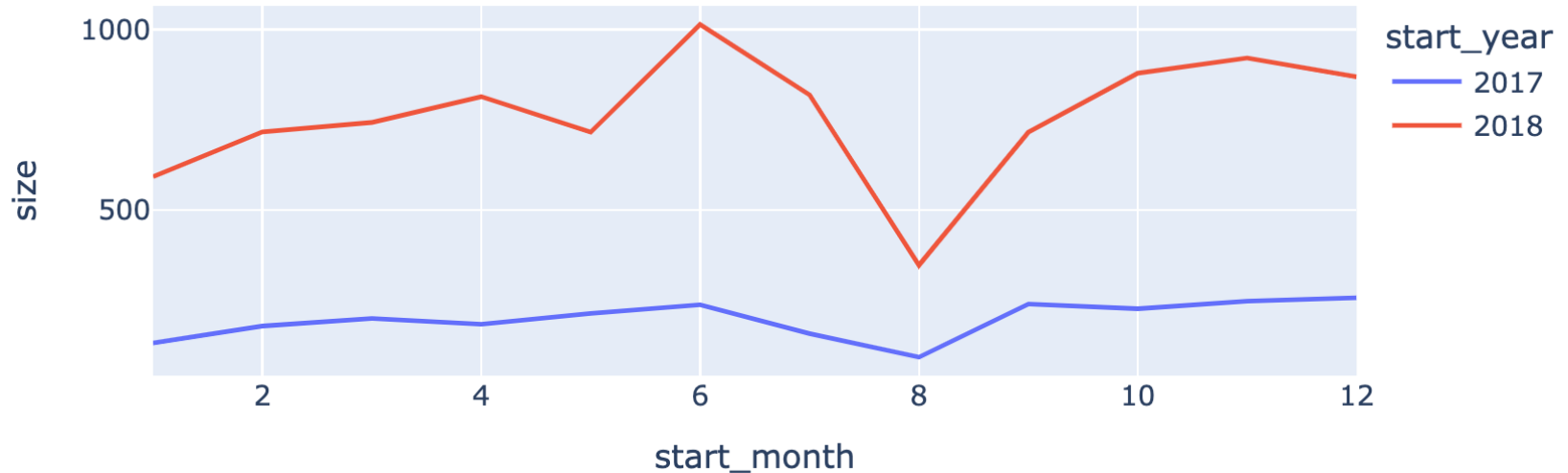
Case Arrival Drift (alternate view, *interactive*)

```
fig = px.histogram(case_log, x='start_month', animation_frame='start_year', range_y=(0,1000))  
fig.update_layout(bargap=0.1)  
fig.show()
```



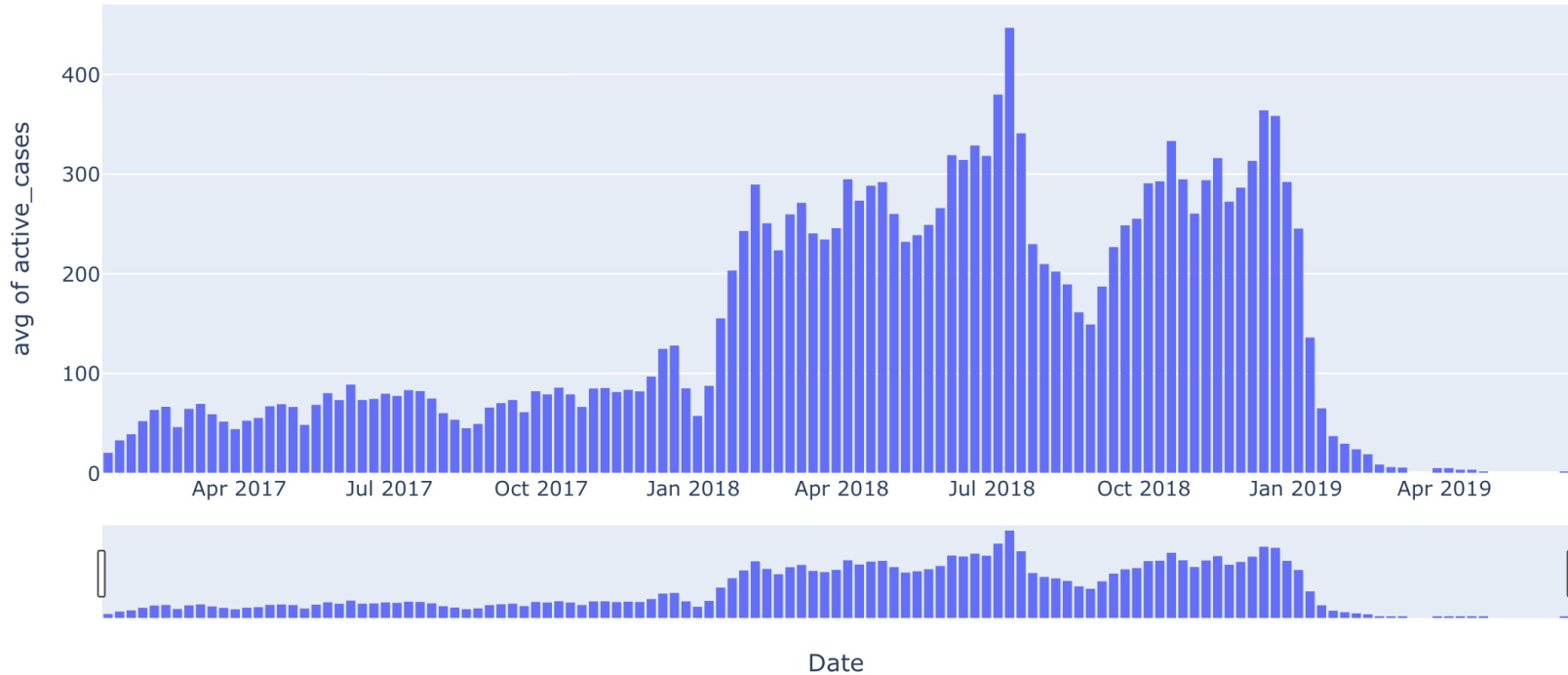
Case Arrival Drift (alternate view, *line plot*)

```
monthly_arrivals = filtered_event_log.groupby(['start_year', 'start_month'], as_index=False).size()  
fig = px.line(monthly_arrivals, x='start_month', y='size', color='start_year', width=700)  
fig.show()
```



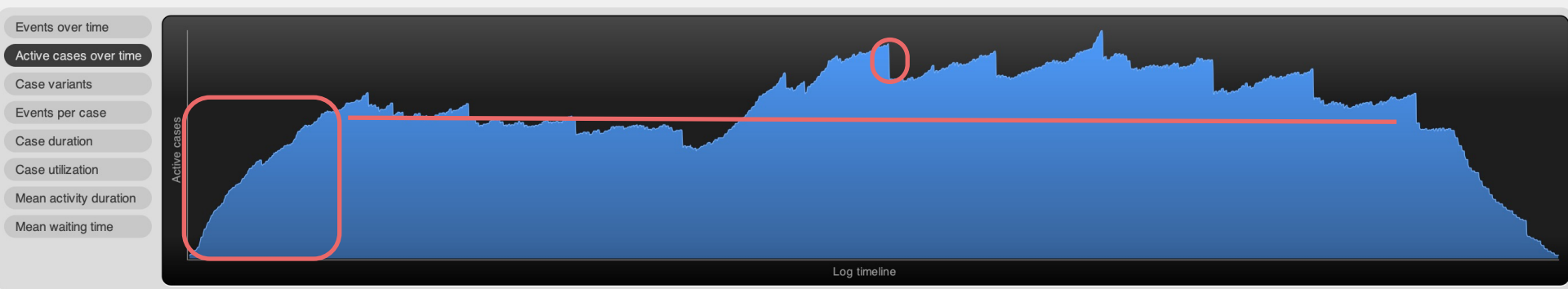
Case Load (*domestic_declarations.ipynb*)

Number of Active Cases



Discussion (3 mins) - Case Load (*Disco*)

Loan Approval Process in a Bank



Discuss the meaning of the three observations with your neighbor.

What is the potential impact of the observation behind the long line?

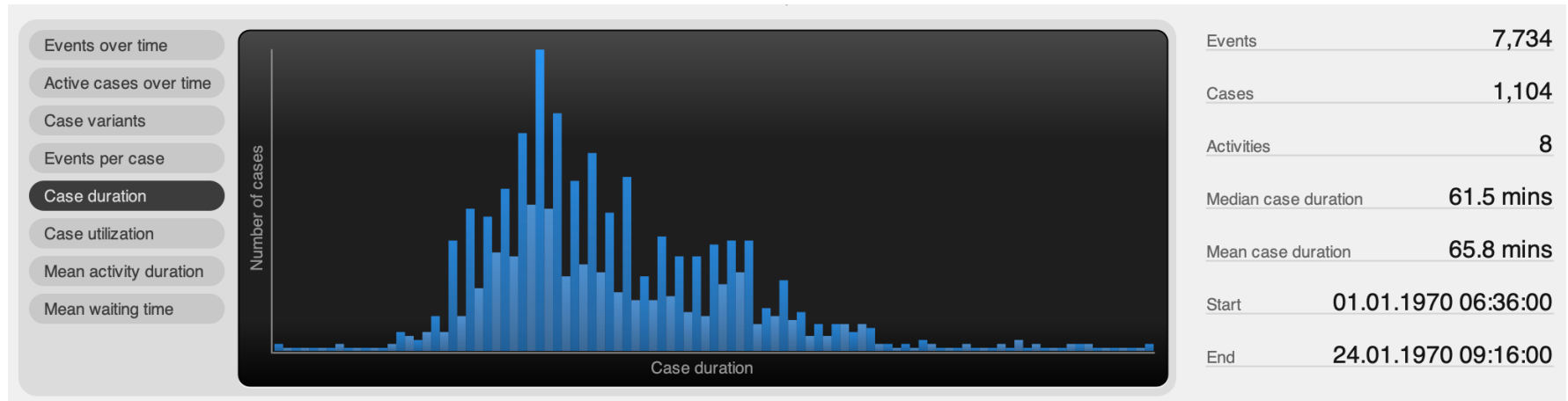
TL;DR

Plotting load for all or specific events as well as cases allows us to see patterns of

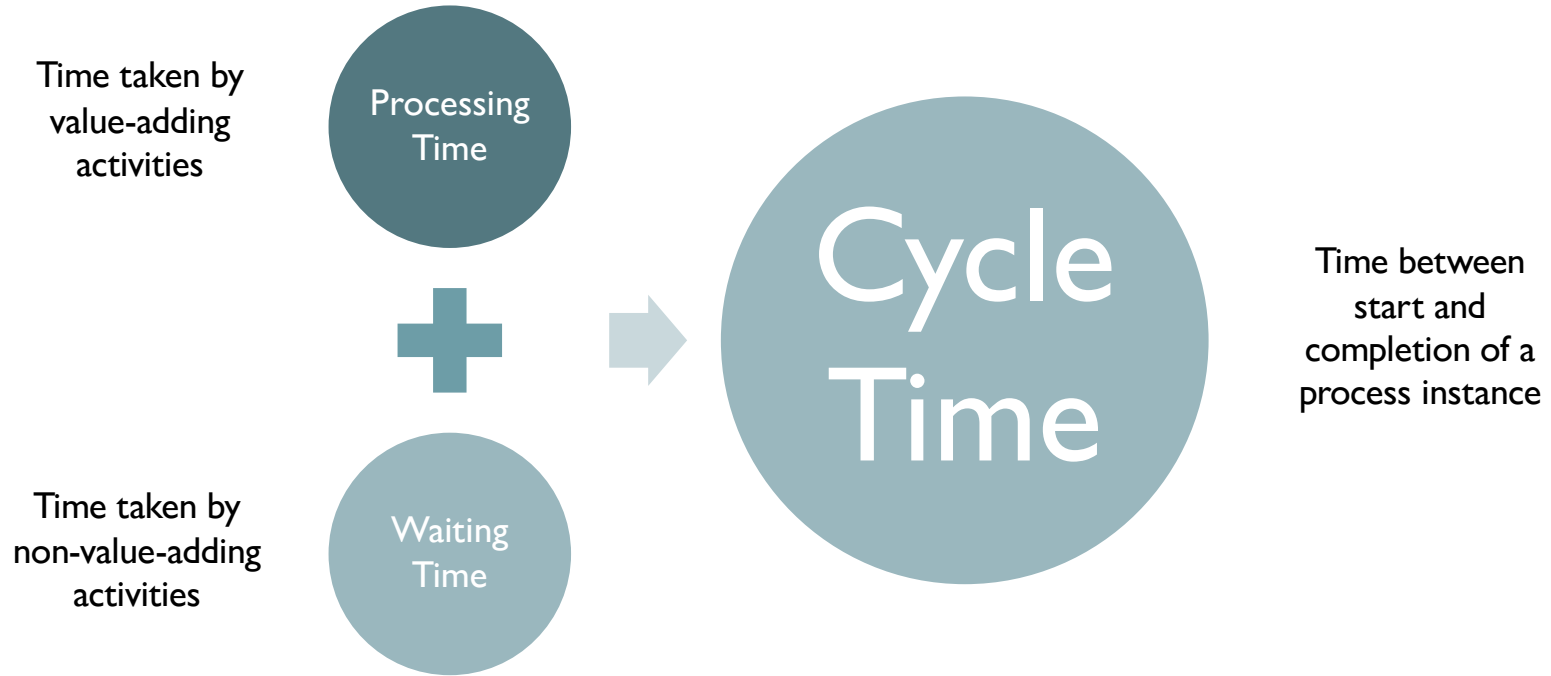
- load distribution
- periodic seasonality patterns
- drift

Cycle Time Analysis

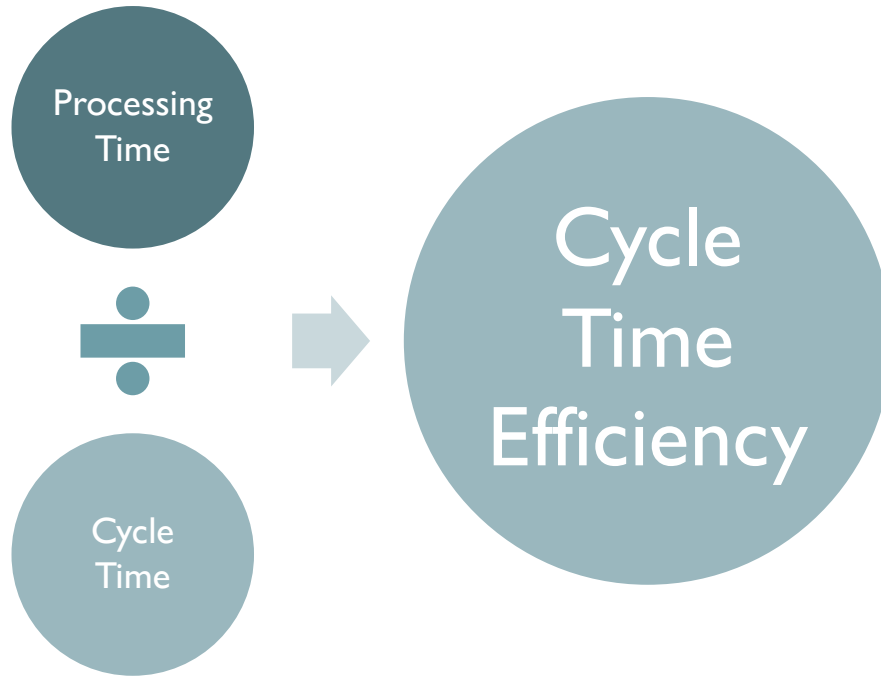
Case Duration a.k.a. Cycle Time



Metrics



Metrics



a.k.a. case utilization

Little's Law



$$\text{case load} = \text{case arrival rate} * \text{cycle time}$$

$\uparrow$
= *throughput* in steady state

Cycle time = average duration of case (e.g. 15 days)

Case arrival rate = the number of arriving cases per time unit on average (e.g. 10 cases per day)

Case load (a.k.a. work in progress) = the number of open cases on average (e.g. 150 cases)

Cycle Time / Bottleneck Analysis

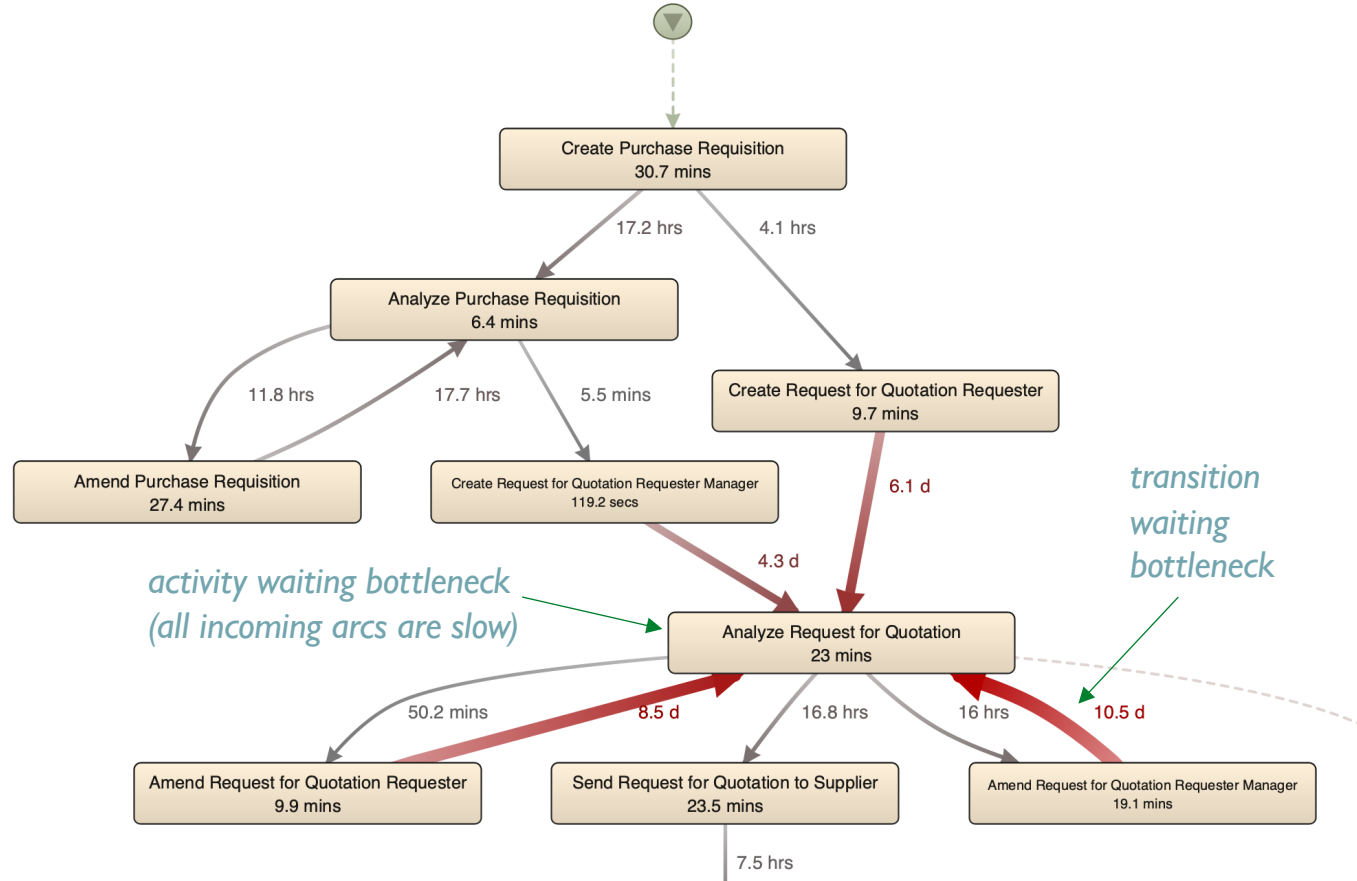
You can breakdown the cycle time

- by **activity**
 - duration
 - waiting time
- by (sequential) **variant**
- by **resource** involvement

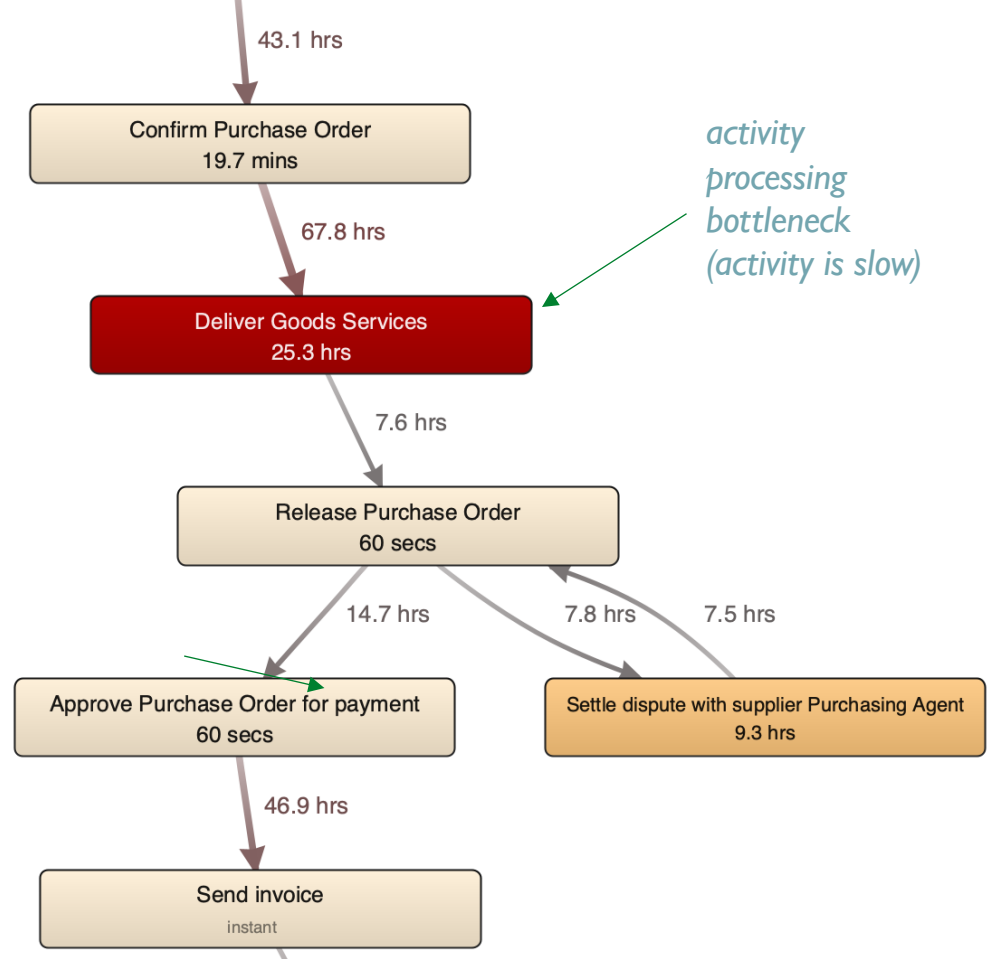
Exercise (5 mins)

Analyze [this event log](#) of a telephone repair process to identify bottlenecks, e.g., by using *Disco*. Which task has the longest waiting time and which one has the longest processing time?

Bottleneck Analysis (Activity Perspective)



Bottleneck Analysis 2 (Activity Perspective)



Relative Case Time (all cases start at time 0)

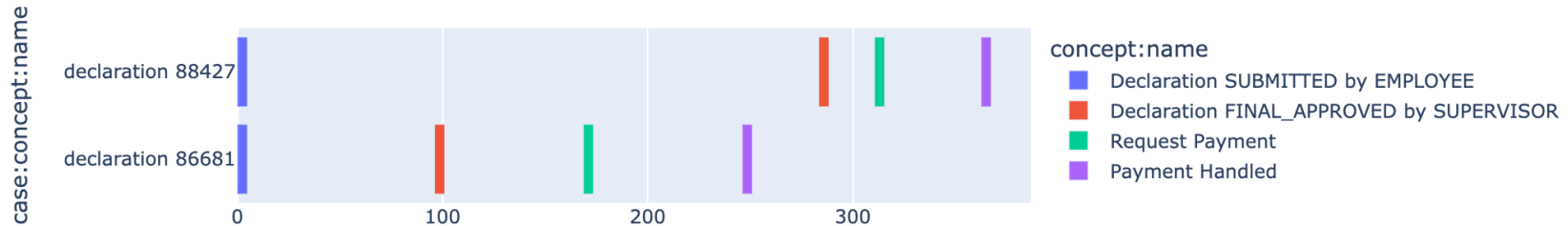
```
event_log['rel_time'] = event_log.groupby(CASE_ID)[COMPLETION_TIME].transform(lambda x: x - x.min())
```

	📁 case:concept:name	📁 concept:name	📅 time:timestamp	📅 rel_time
0	declaration 86791	Declaration SUBMITTED by EMPLOYI	2017-01-09 09:49:50+00:00	0 days 00:00:00
1	declaration 86791	Declaration FINAL_APPROVED by SU	2017-01-09 11:27:48+00:00	0 days 01:37:58
2	declaration 86791	Request Payment	2017-01-10 09:34:44+00:00	0 days 23:44:54
3	declaration 86791	Payment Handled	2017-01-12 17:31:22+00:00	3 days 07:41:32
4	declaration 86795	Declaration SUBMITTED by EMPLOYI	2017-01-09 10:26:14+00:00	0 days 00:00:00
5	declaration 86795	Declaration APPROVED by PRE_APPF	2017-02-22 10:29:21+00:00	44 days 00:03:07
6	declaration 86795	Declaration FINAL_APPROVED by SU	2017-02-23 08:14:45+00:00	44 days 21:48:31
7	declaration 86795	Request Payment	2017-03-06 14:07:25+00:00	56 days 03:41:11
8	declaration 86795	Payment Handled	2017-03-13 17:30:59+00:00	63 days 07:04:45

Compare Two Cases

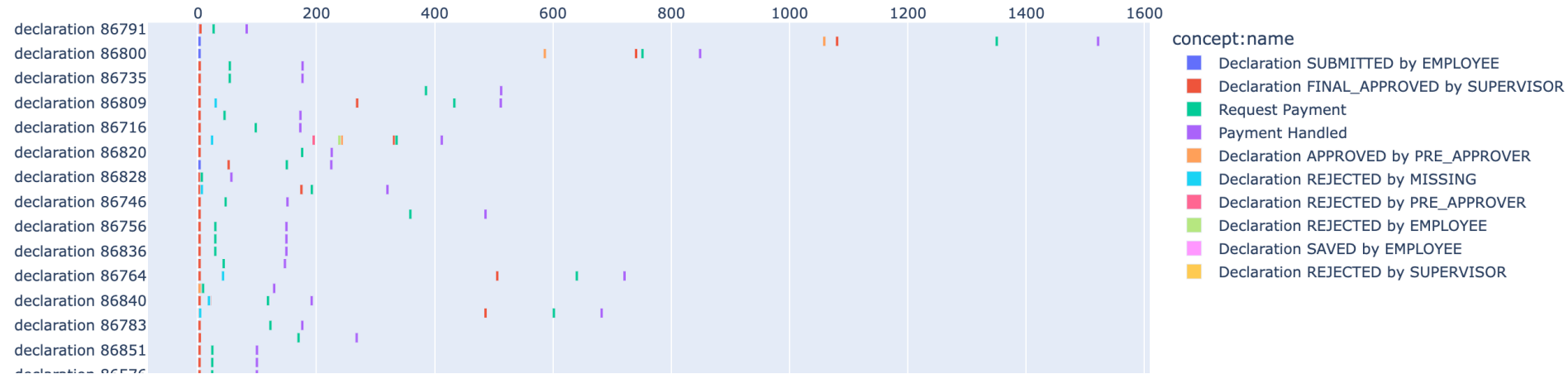
work with integer; choose appropriate unit

```
event_log['rel_time_hours'] = event_log['rel_time'] / pd.Timedelta(hours=1)
bar_size = 5
filtered_event_log = event_log[event_log['case:concept:name'].isin(
    ['declaration 86681', 'declaration 88427']) ]
fig = px.bar(filtered_event_log,
             base = 'rel_time_hours',
             x = len(filtered_event_log)*[bar_size],
             y = 'case:concept:name',
             color = 'concept:name',
             orientation = 'h', height=250, width=1000)
fig.show()
```



Dotted Chart (relative time)

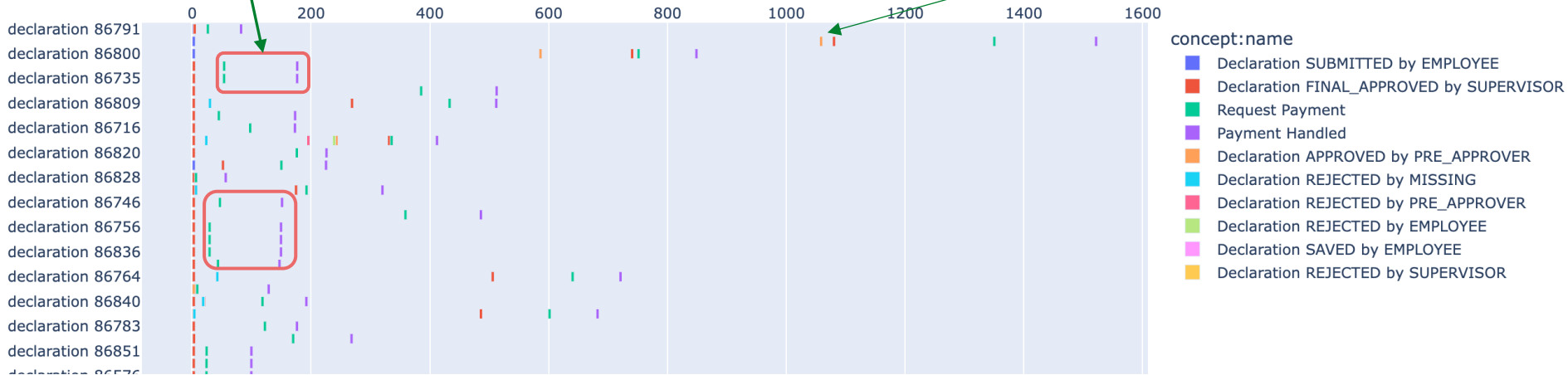
What can we see?



Dotted Chart (relative time)

consistent distances between Request Payment and Payment Handled?

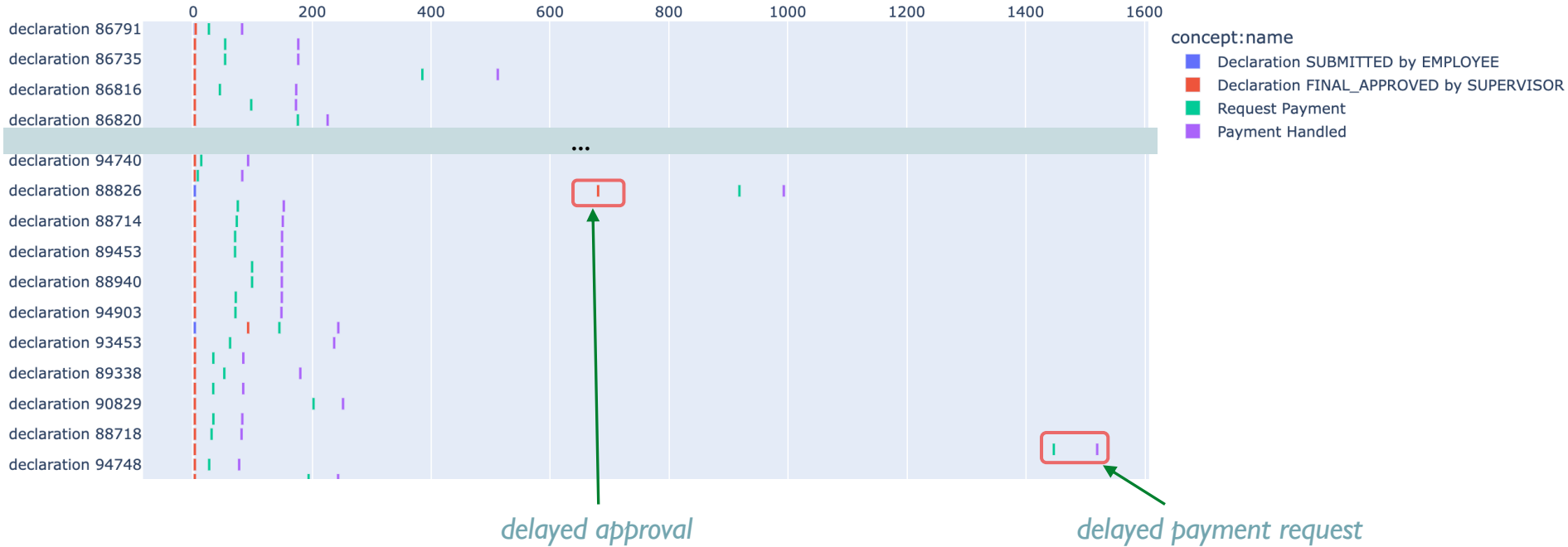
outlier?



Dotted Chart (a single sequential variant)



Dotted Chart (a single sequential variant, ctd.)



```

case_log['selected'] = case_log['seq_variant'].apply(
    lambda x: x == ['Declaration SUBMITTED by EMPLOYEE',
                    'Declaration FINAL_APPROVED by SUPERVISOR',
                    'Request Payment',
                    'Payment Handled'])

event_log['selected'] = event_log.apply(
    lambda row: case_log.at[row['case:concept:name'], 'selected'],
    axis=1)

filtered_event_log = event_log[event_log['selected'] == True]
filtered_event_log = filtered_event_log[:600]

bar_size = 5
fig = px.bar(filtered_event_log,
             base = 'rel_time_hours',
             x = len(filtered_event_log)*[bar_size],
             y = 'case:concept:name',
             color = 'concept:name',
             orientation = 'h',
             height=2000)

fig.update_yaxes(autorange='reversed')
fig.update_layout(xaxis={'side': 'top'}, yaxis={'side': 'left'})
fig.show()

```

select a specific sequential variant

case log lookup

reduce size of event log

stacked horizontal bar chart

TL;DR

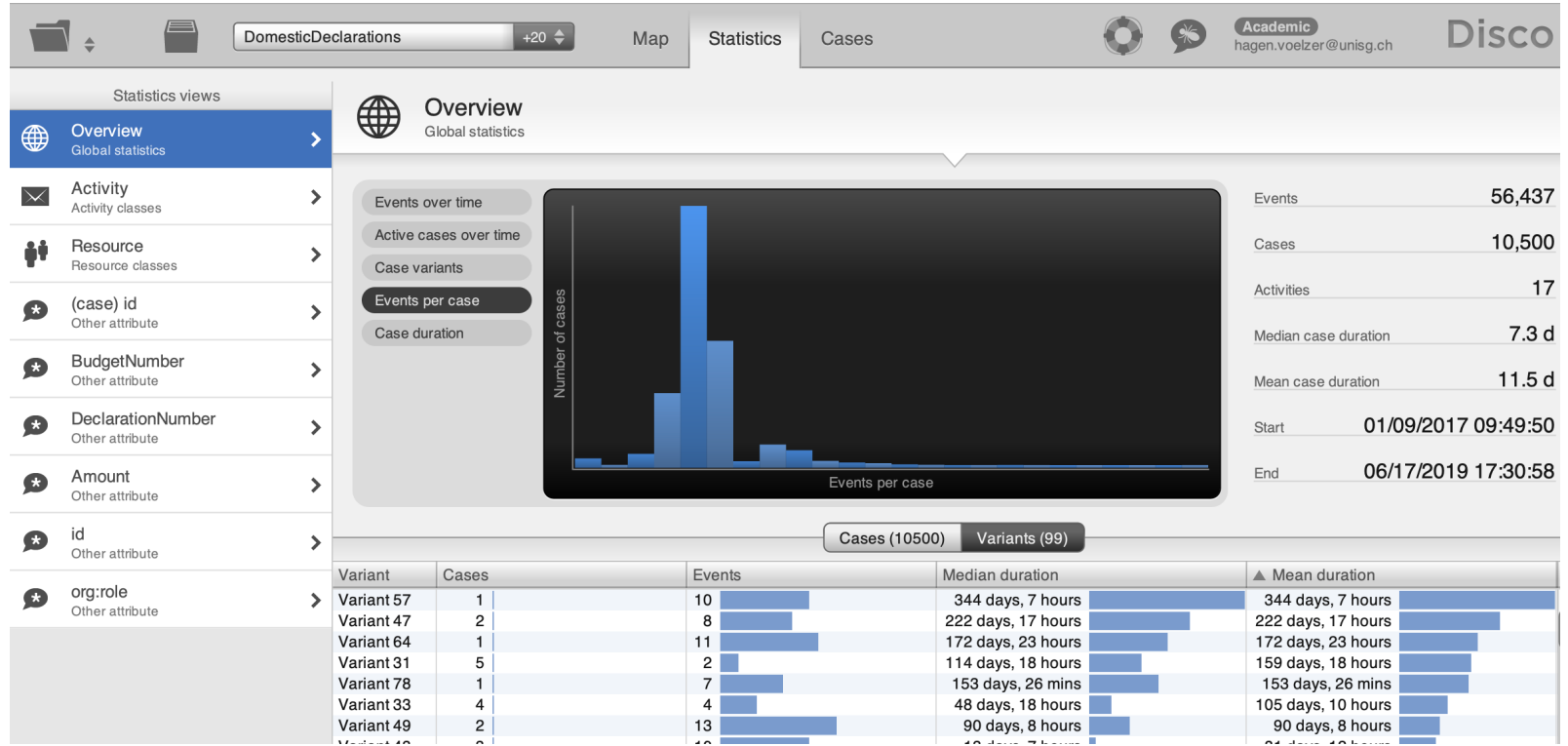
The relative time dotted chart allows us to discover *indications* of:

- delay variation and consistency in all or just similar cases (e.g. cases of the same sequential variant)
- outlier delays

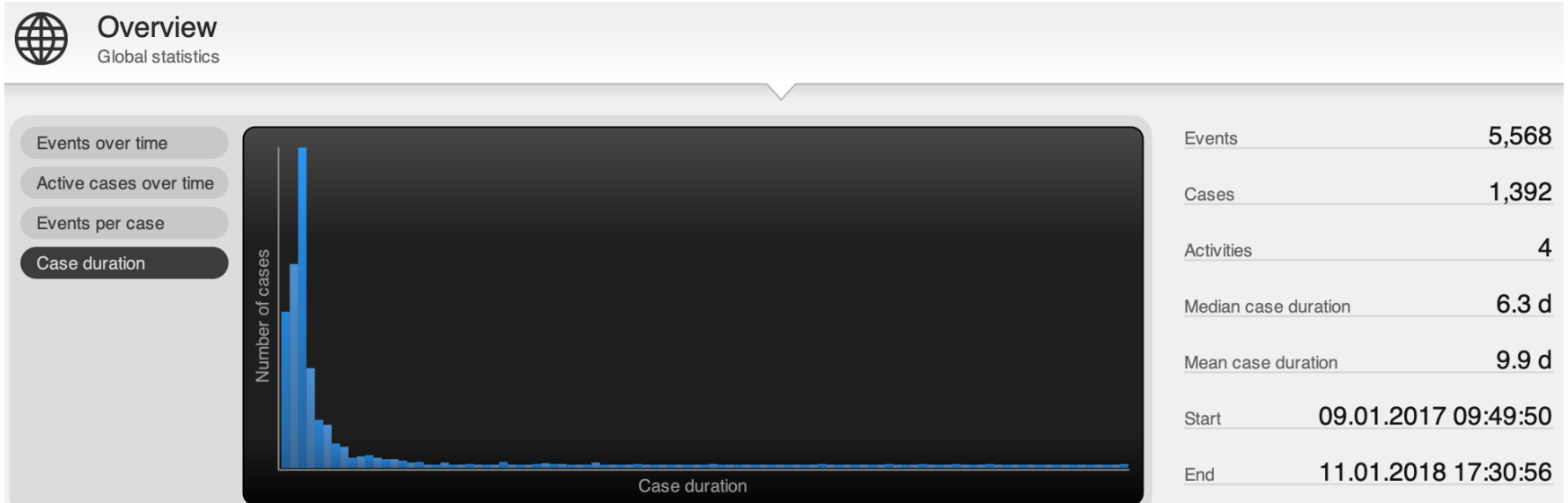
modulo the number of cases we are able to fit into the chart.

Relative time could be generalized by aligning cases on any given event not just on the start event.

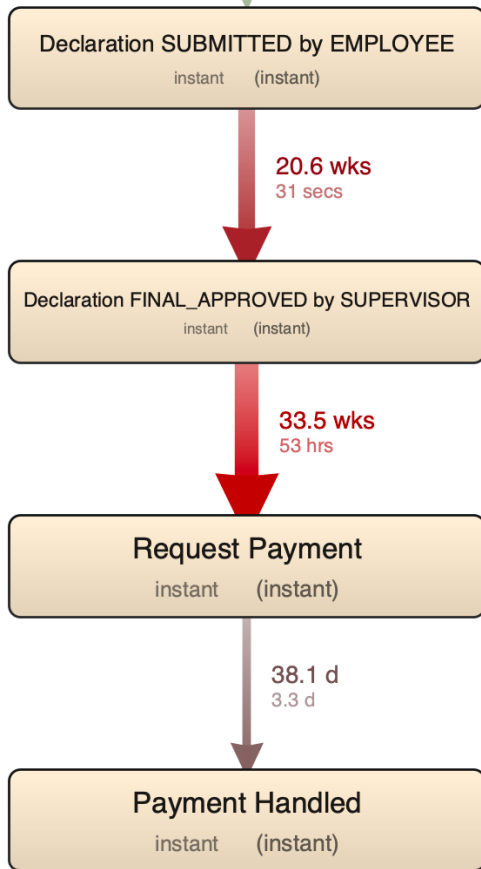
Cycle Time by Variant



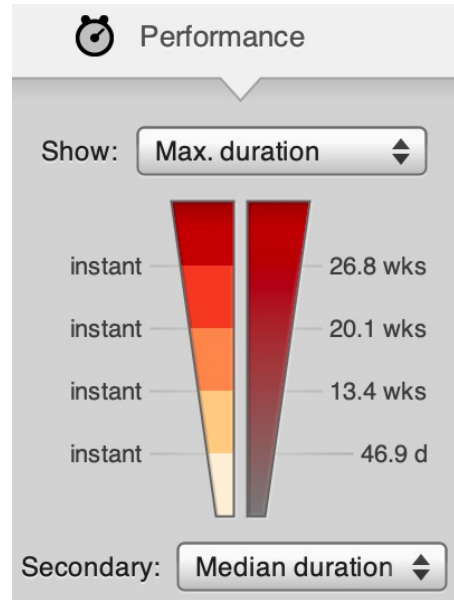
Disco: Case Duration (under variant filter: only variant no. 3, domestic declarations)



See the main performance and extent of outliers of a given variant before breaking it down.

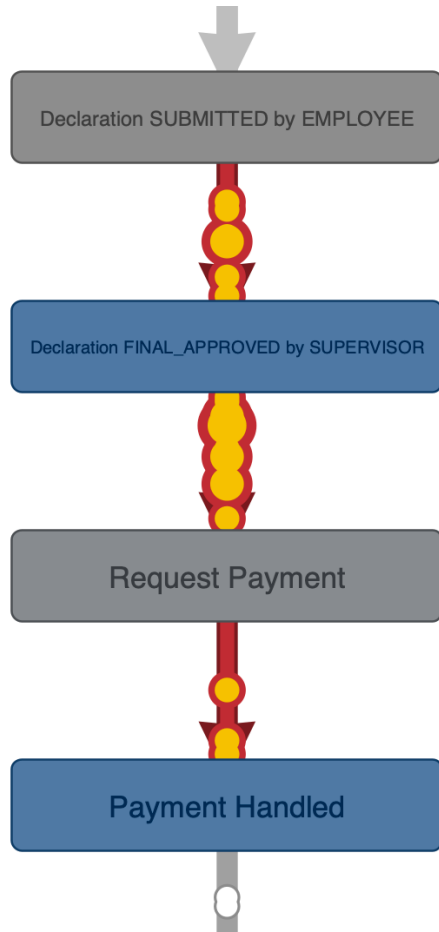


Disco: Performance Annotated DFG (under the same variant filter)



- Choose between 5 aggregates
- color coding shows bottlenecks with respect to main selected aggregate
- None of the aggregates gives clear picture of the underlying distribution
 - missing histograms

Disco: Performance Animation DFG (under the same variant filter)



- can be run with absolute time or relative time
- bottlenecks are visible as increased load over time (accumulation of tokens) for absolute time
 - shows delay dependent on absolute time, hence on load

Performance Animation

runs also in branching DFG

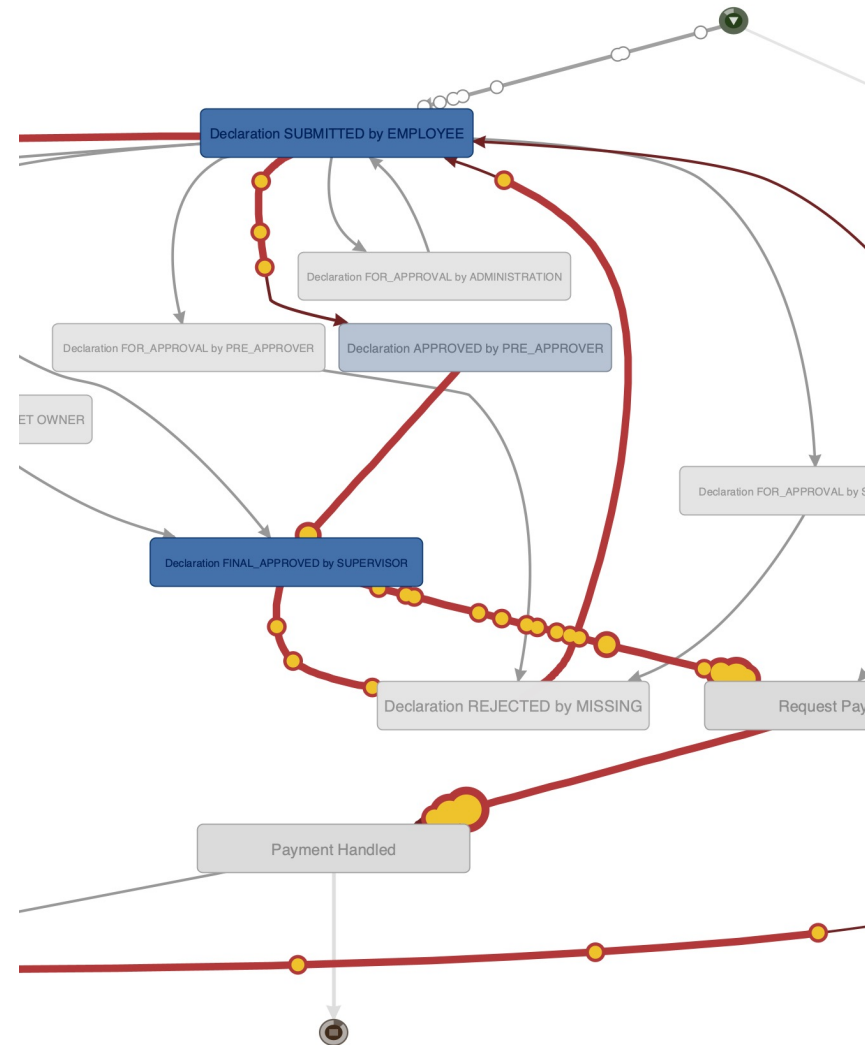
- *more than one sequential variant selected*

Relative Time

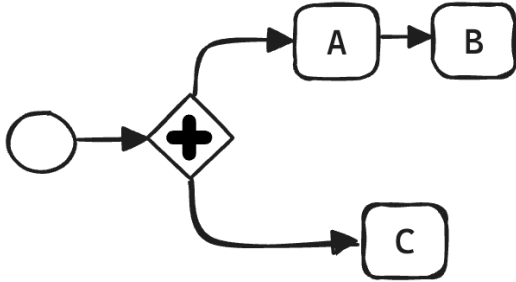
- similar to dotted chart with all cases
- information about **variants**
- information about cycle time distribution and **-outliers**

Absolute Time

- reflects **actual arrival patterns**
- better for detecting **bottleneck** occurrences



Inter-Activity Delay in the Performance DFG under Concurrency



This process fragment can show up in the log often as sequence

A C B

In the corresponding performance DFG, the (aggregated) delay between A and B will not show up correctly.

It is better to visualize a user-selected delay (A,B) directly (*or filter out concurrency first, e.g., by filtering a single business object lifecycle*).

Towards Delay Histograms – Computing Delays

```
def convert_name(name):  
    return '_' + name.split(' ')
```



```
def add_start_time(event_log, case_log, activity):  
    filtered_event_log = event_log[event_log['concept:name']==activity]  
    col_name = convert_name(activity) + '::start'  
    case_log[col_name] = filtered_event_log.groupby('case:concept:name')['rel_time'].first()  
    return case_log
```

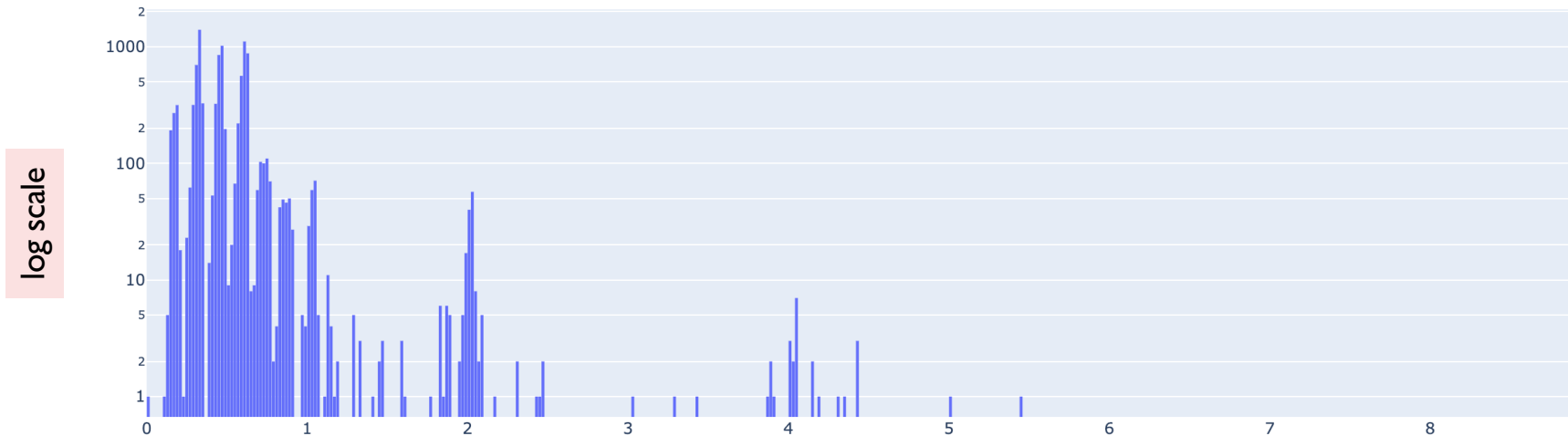


```
def add_distance(case_log, activity1, activity2):  
    activity_name1 = convert_name(activity1)  
    activity_name2 = convert_name(activity2)  
    col_name = activity_name1 + ':' + activity_name2 + '::dist'  
    case_log[col_name] = case_log[activity_name2+'::start'] - case_log[activity_name1+'::start']  
    return case_log
```

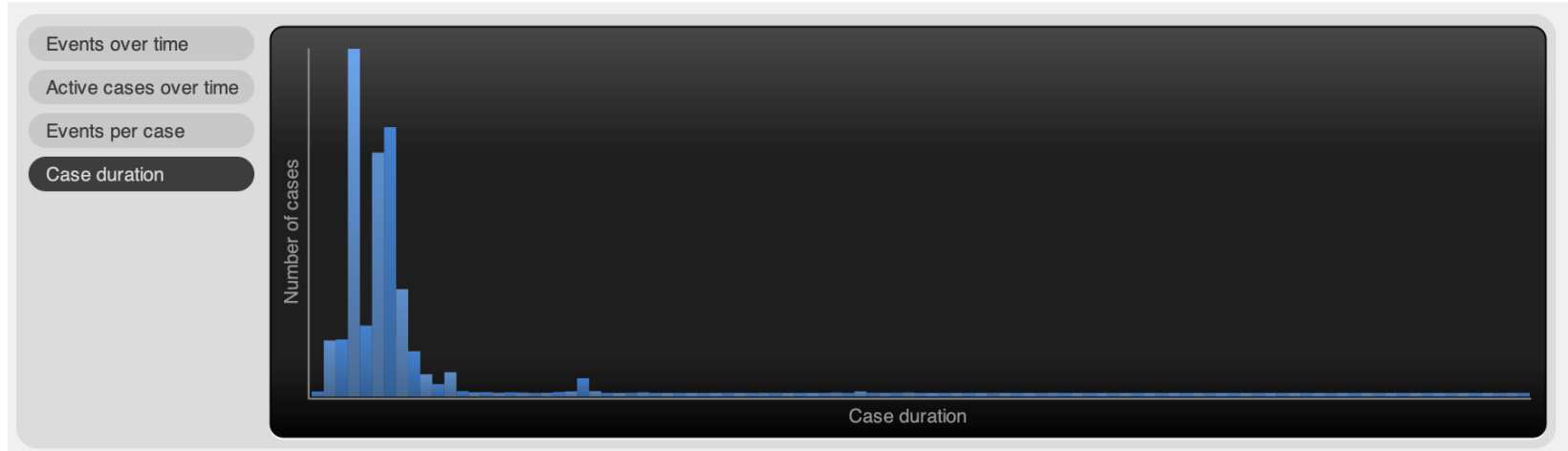
- compute delay between any pair A,B of activities, not just subsequent ones
- assume here we have one timestamp only per event (otherwise adjust the code)

Delay Histogram *(study distribution of a specific delay)*

```
case_log = add_start_time(event_log, case_log, 'Request Payment')
case_log = add_start_time(event_log, case_log, 'Payment Handled')
case_log = add_distance(case_log, 'Request Payment', 'Payment Handled')
fig = px.histogram(case_log['Request_Payment:Payment_Handled::dist'] / pd.Timedelta(weeks=1), log_y=True)
```



Delay Histogram in Disco (workaround)



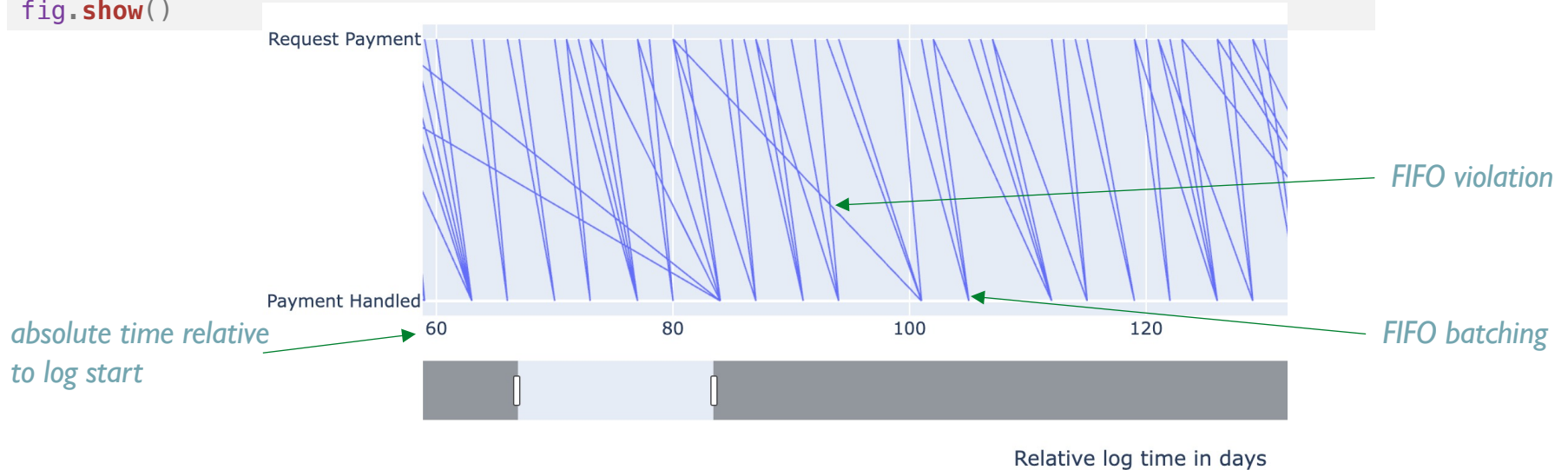
- Filter the event log down to include only *Request Payment* and *Payment Handling*
 - Plot *case duration*
- Make sure the order of the two events is (essentially) unique
 - otherwise distinguish both ordering cases with a *follower* filter

Performance Spectrum

```
from event_enricher import event_add_relative_log_time
from visualizations import performance_spectrum
```

```
event_log = event_add_relative_log_time(event_log)
event_log['rel_log_time'] = event_log['rel_log_time'].dt.days
```

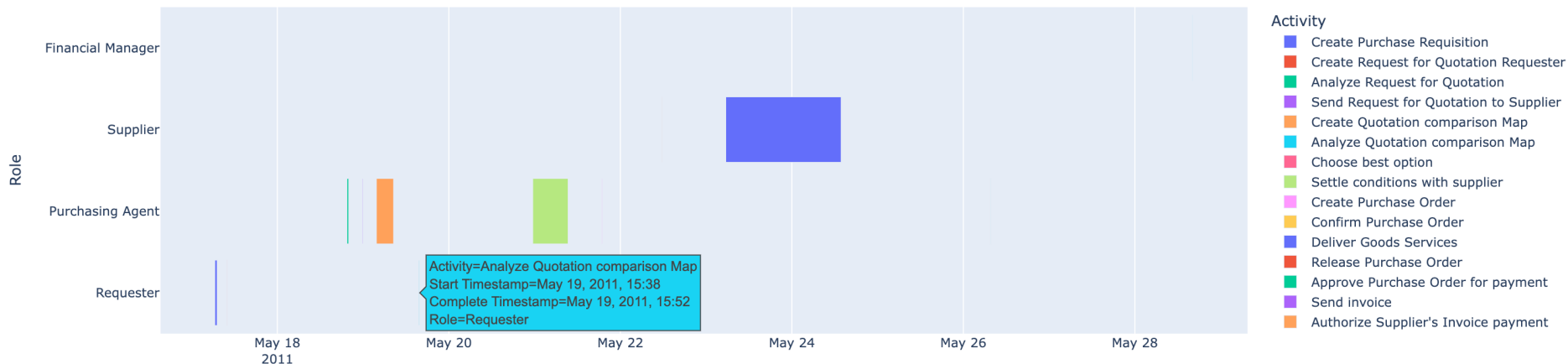
```
fig = performance_spectrum(case_log, event_log, 'Request Payment', 'Payment Handled')
fig.show()
```



Outlook: Timeline with Durations (Purchasing-Example)

```
filtered_event_log = event_log[event_log['Case ID'] == 940]
fig = px.timeline(filtered_event_log,
                 x_start=(filtered_event_log['Start Timestamp']),
                 x_end=filtered_event_log['Complete Timestamp'],
                 y='Role',
                 color='Activity')
fig.show()
```

UX Problems?



Outlook: Richer Flow Information

- events in xes event logs typically do not have a *start* and a *complete* timestamp
- rather, *start* and *complete* are separate events
- there can even be more *lifecycle transitions*, e.g.
 - *schedule*
 - *resume*
 - *suspend*
 - *withdraw*
- those can occur in logs of *workflow* management systems and *workforce* management systems
 - this allows us to distinguish *effort* from *delay*

EventID	lifecycle:transition
Missing: 0 (0%)	Missing: 0 (0%)
Distinct: 1202267 (100%)	Distinct: 7 (<1%)
1202267 Distinct values	complete: 40%
	suspend: 18%
	schedule: 12%
	Other: 30%
Workitem_338353106	suspend
Workitem_1522720538	resume
Workitem_313764426	suspend
Workitem_1377364007	resume
ApplState_1181660825	complete
OfferState_1494429296	complete
Workitem_72878931	complete
Application_428409768	complete
ApplState_105207793	complete
Workitem_1185806608	schedule
Workitem_835967796	withdraw
Workitem_755889823	schedule
ApplState_1727723699	complete
Workitem_62767814	start

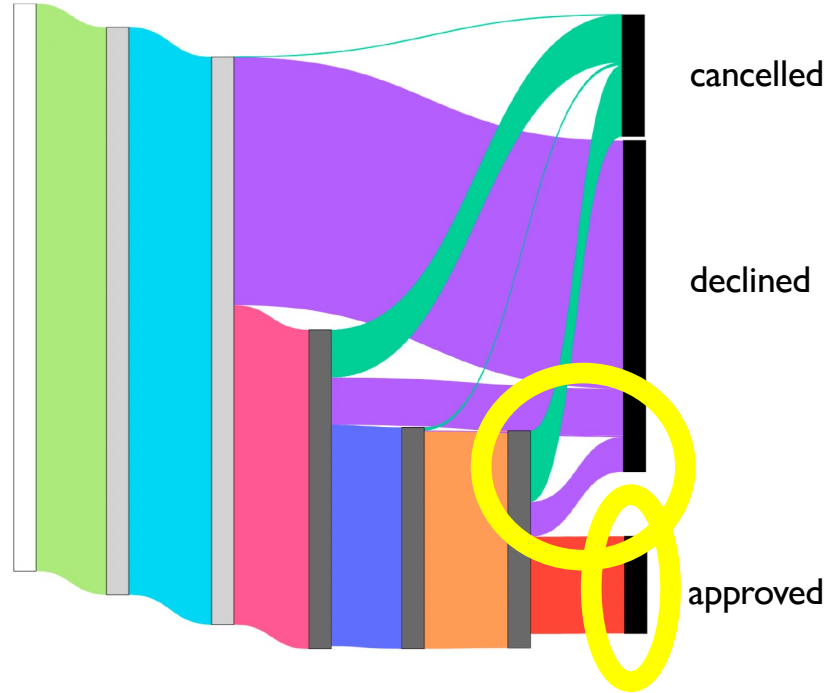
Shift-Left Analysis

(a.k.a. Over-Processing Analysis)

LCM (loan approval)

Activities

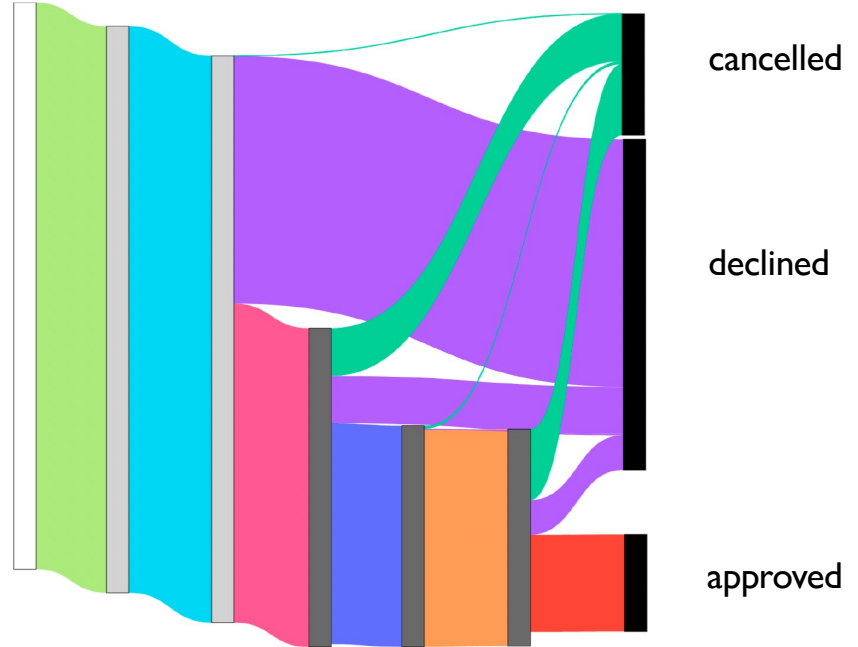
- A_ACCEPTED
- A_APPROVED
- A_CANCELLED
- A_DECLINED
- A_FINALIZED
- A_PARTLYSUBMITTED
- A_PREACCEPTED
- A_SUBMITTED



Shift-left (a.k.a. Overprocessing) Analysis

Knockout process =

- essentially binary outcome *success* or *failure*
 - we may distinguish different failures
- multi-stage filtering
 - each filter incurs a cost
 - only succeeding cases create value (ROI)



Knockout Process Examples

- Loan Approval
- Road Fines Management
- Travel reimbursement
- Hiring
- Incident Management
- Complaint Management
- Quality control
- Fraud detection
- Intrusion detection
- ...

Shift-Left Analysis

Can some of the cases that are knocked out late in the process be knocked out earlier (*shifted left*)?

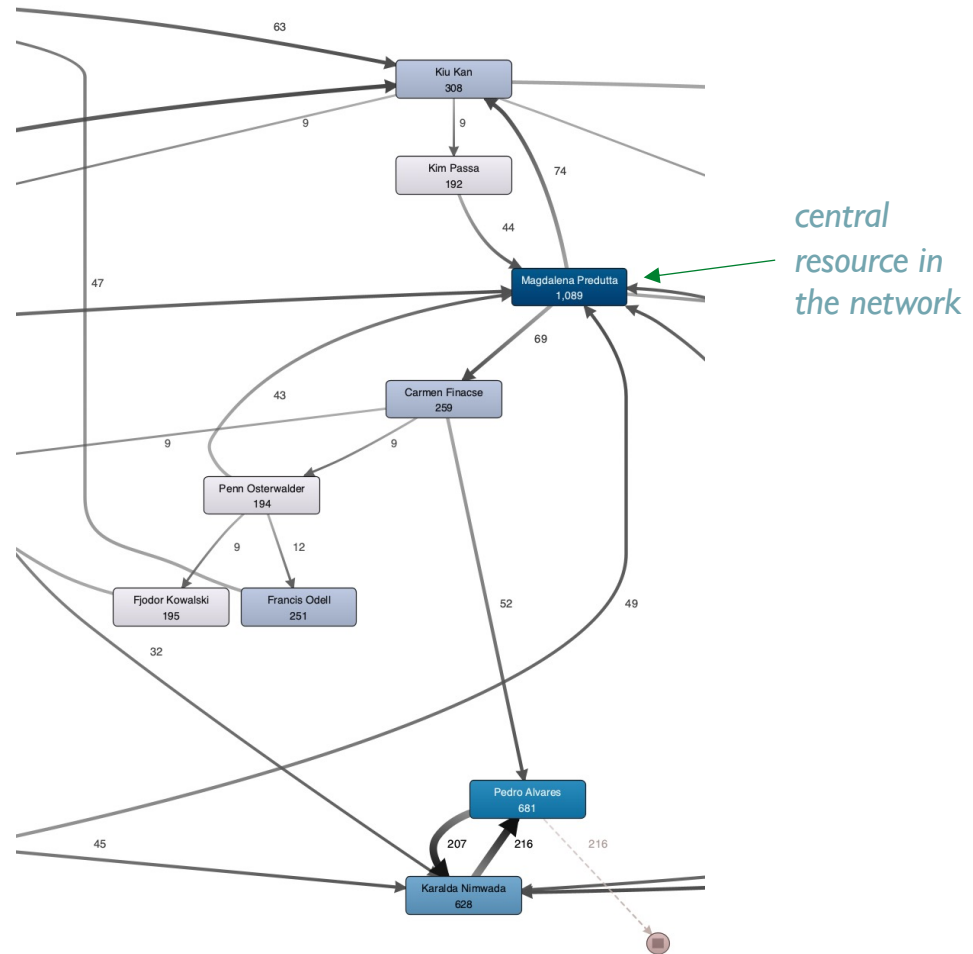
- find individual cases that are over-processed
- characterise such cases to quantify the waste and estimate the gain from changing the process
 - in a call center, tickets that are routed to specialist but could be knocked out by a chat bot
 - incidents in a data center that could be detected with proper monitoring, predictive maintenance
 - job applications that can be knocked-out with automated inspection
 - bugs that are found in production that could have been detected during testing
 - ...

Additional Techniques and Summary

Option: Breakdown by Resource Utilization

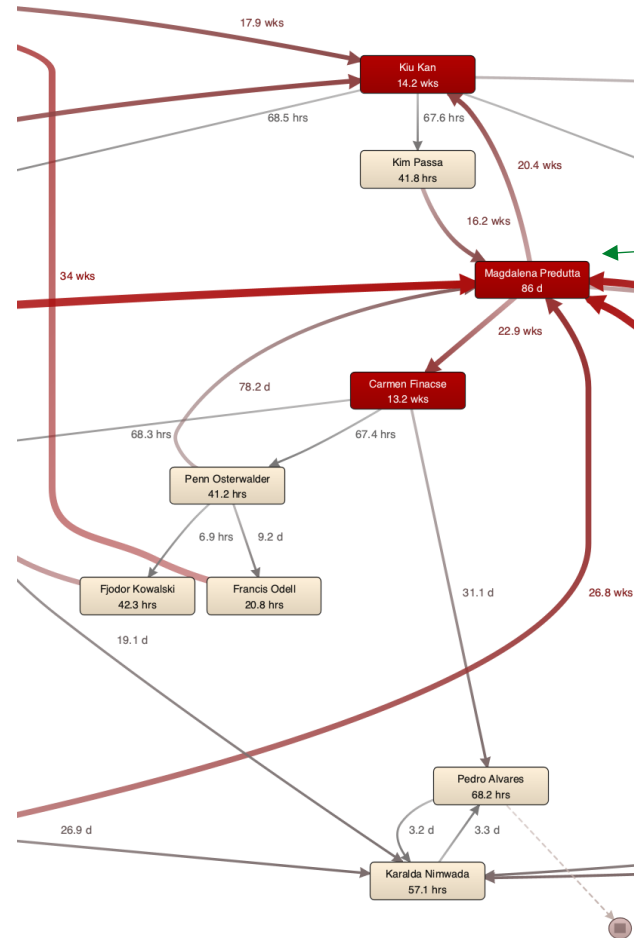
(purchase example)

- Analyze resource involvement if data is available
- In *Disco*: use *Resource* as *Activity* attribute
 - for an xes file, you can export the event log as csv and then re-import
- Study *handover* patterns
 - network centrality (*robust against failure?*)
 - handover minimization
 - resource bottlenecks

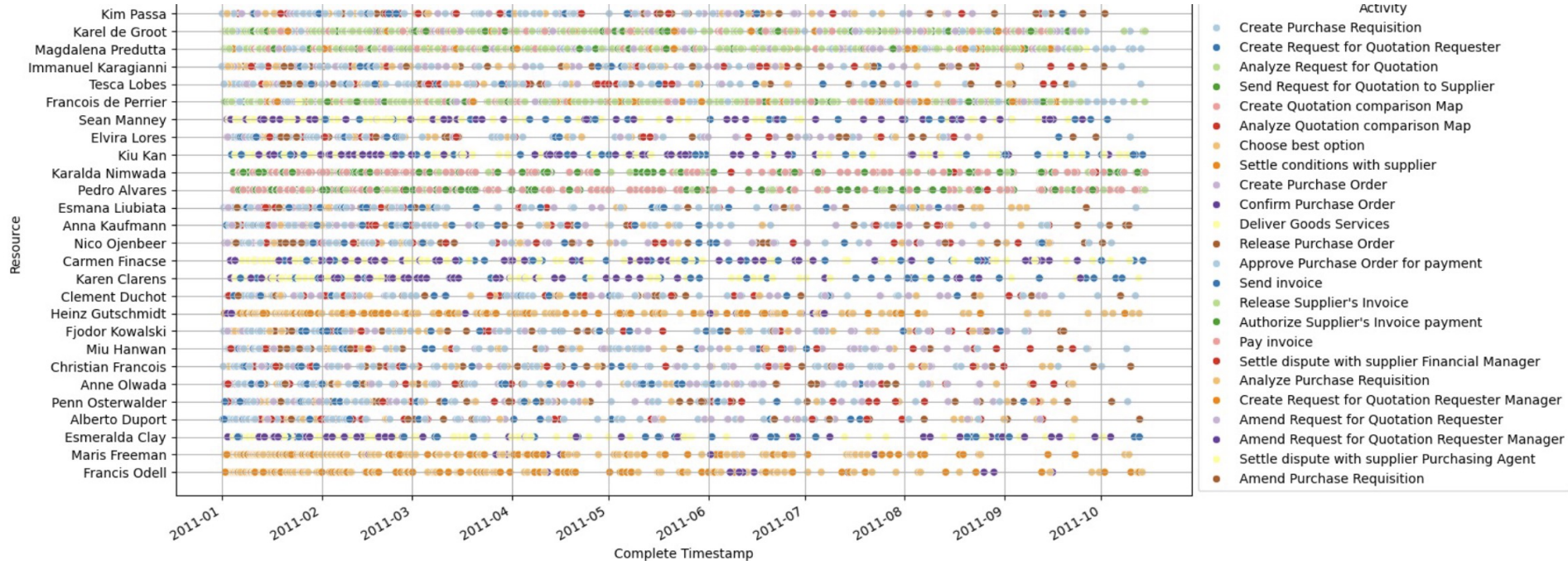


Option: Breakdown by Resource Utilization (ctd.)

- Study
 - resource over- and under-utilization
 - resource bottlenecks
- Use *Disco* animation



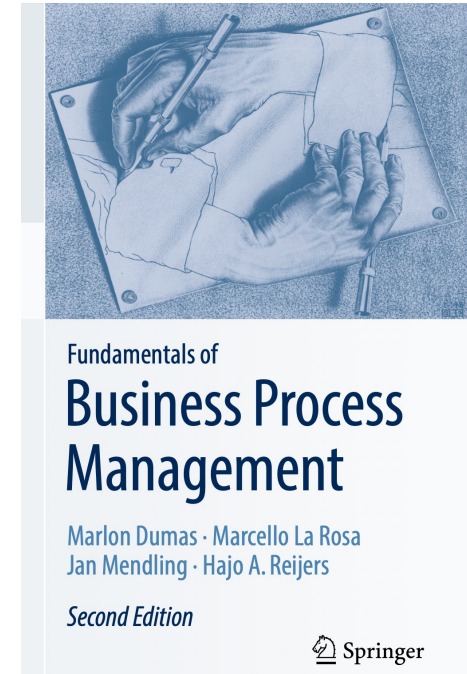
Breakdown by Resource – Dotted Chart *(purchase example)*



Further Reading

Dumas et al.: [Fundamentals of Business Process Management](#), Springer 2018

- Chapter 11.5



Analysis Techniques Review

Rework Analysis Identify defects/errors and rework waste

Workload & Demand Analysis

- Analyze demand & *inventory* (plot/analyze case arrival, departure and activeness over time, inventory is similar, break down for events, study seasonality patterns)
- Similar: Analyze *resource workload* over absolute time (if data is available), Identify overused vs. underused resources/idleness waste
- Use filtering to focus on cases with highest cycle/processing/waiting times.

Bottleneck Analysis

- Identify *activity bottlenecks* (high-effort activities = long processing times)
- Identify *waiting bottlenecks* (slow handoffs and resource-capacity bottlenecks, long times between subsequent activities)
- Identify *resource bottlenecks* (instead of mapping activities, map resource to analyze processing time and waiting time per resource)
- Use animation to visualize build-up of bottlenecks

Assignment 1: Tasks from Session 5 Part 1 (Rework Analysis)

1. Do a *rework analysis* on the original log.
 - a) Find and quantify rework.
 - b) Inspect cases with small and high number of repetitions and formulate for each inspected case a hypothesis why the rework occurred.

Assignment 1: Tasks from Session 5 Part 2 (Load Analysis)

2. Investigate the following distributions: Start by writing down one or more hypotheses about the distribution, then investigate. Report your most striking or interesting findings, present a suitable visualization for each finding (*could be multiple findings per visualization*), and give an interpretation for each finding.
 - a) the *arrival* of cases over time. (*Investigate seasonality at different levels of granularity and drift year by year.*)
 - b) the *closure* of cases over time
 - c) the *activity load* for *Send for Credit Collection* and *Send Appeal to Prefecture* over the time.

Assignment 1: Tasks from Session 5 Part 3 (Delay Analysis)

3. Study and briefly describe the distribution of case duration, i.e., cycle time, for (a) all cases *sent to credit collection* and (b) all *fully paid* cases.
4. Consider the following subprocess given as a sequence S of activities:
 $S = ['\text{Create Fine}', '\text{Send Fine}', '\text{Insert Fine Notification}', '\text{Add penalty}', '\text{Send for Credit Collection}']$
Study the distribution of the delay between each pair of subsequent activities in the sequence, i.e., 4 distributions.
 - a) Write down at least one interesting observation for each distribution.
 - b) What changes would you propose to improve the performance of this subprocess? Argue why you propose the improvements.
 - c) The Italian law prescribes a deadline by when the *Send Fine* letter has to be sent. There is a deadline for letters sent to offenders living in Italy and another deadline for those living abroad. Assuming that, in the vast majority of cases, the letter is sent within Italy, what deadline does the distribution suggest existed for these cases? Briefly argue why.
 - d) How many major peaks (a.k.a. modes) do you see in the distribution of the *transit time* of the *Send Fine* letter? Give an interpretation for each of those peaks.
5. Use the *performance spectrum visualization* to refine your findings from Task 2c) (this Session, Part 2). Describe your refined findings.

Assignment 1: Tasks from Session 5 Part 4 (Concept Drift)

6. The *initial fine amount* of a case is the amount associated with the *Create Fine* event of that case. Study how the initial fine amount evolve over time for offenses under a fixed *article* of the law. Build an *interactive* visualization that shows a suitable visualization for an *article* chosen by the user. Describe your findings for an example *article*.
7. Filter the case log to retain all cases that start after Sept 1, 2010. Re-draw the distribution of the delay between *Create Fine* and *Send Fine*. Describe the difference with the corresponding distribution for the full log in Task 4 (this Session, Part 3).